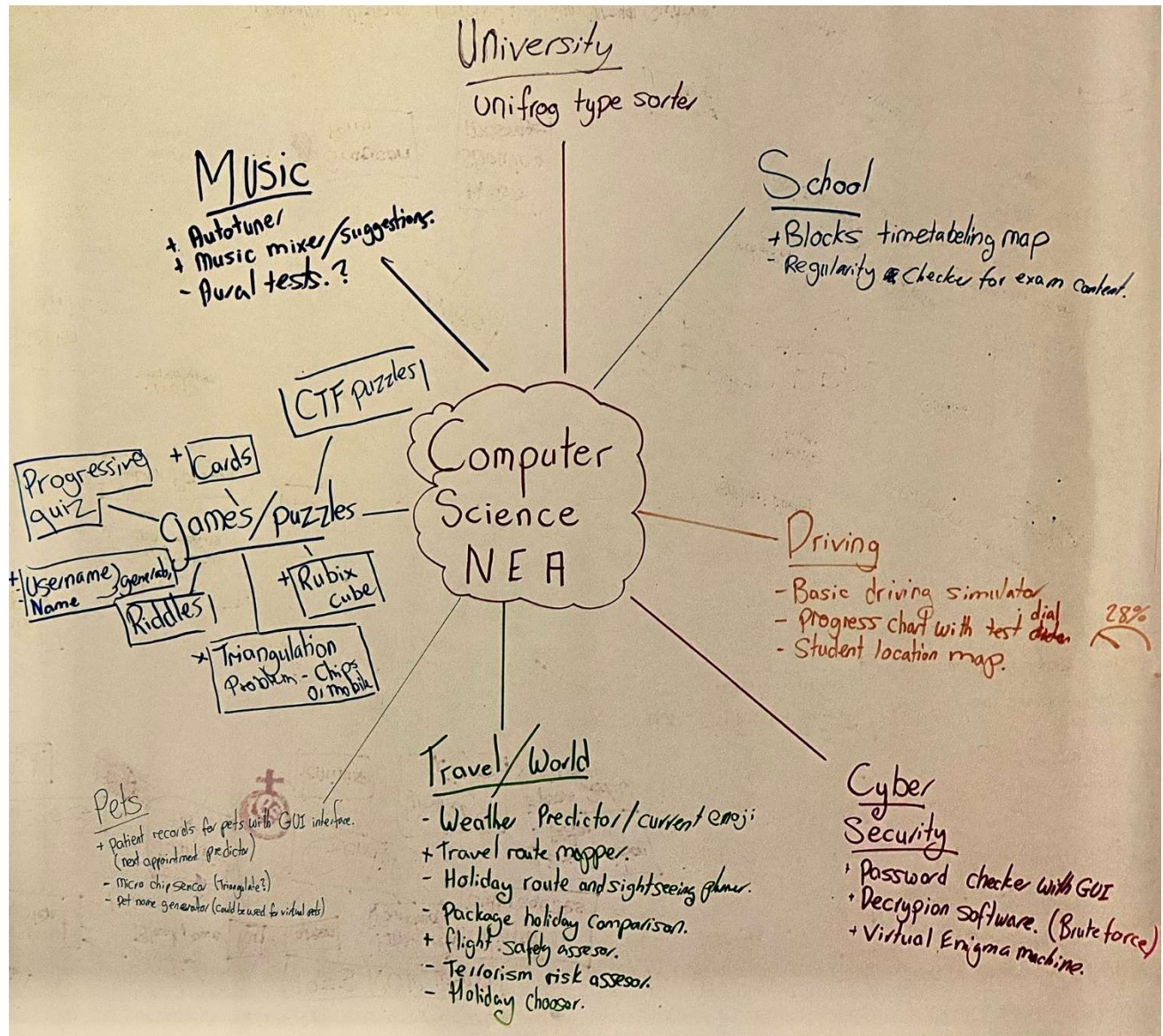


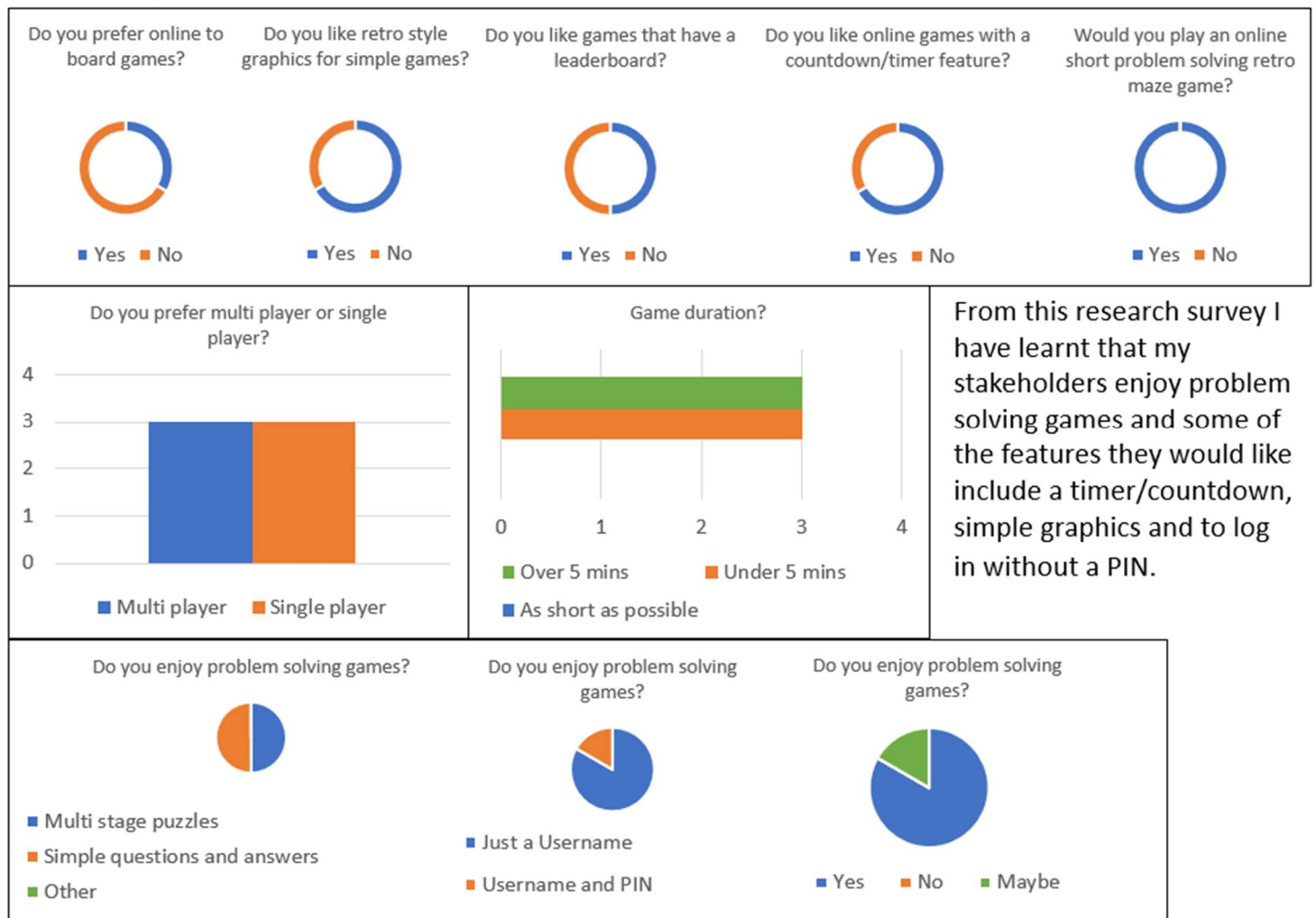
## 40. Appendix A: Brainstorm for Project Ideas

Image of original brainstorm



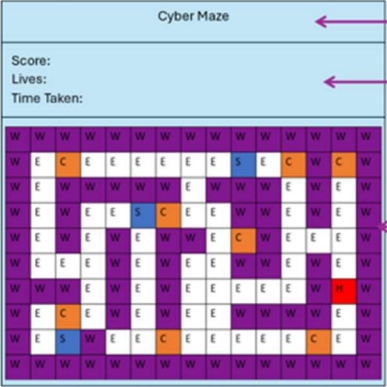
## 41. Appendix B: Stakeholder Research

### Sixth Form Friends: Responses to a survey.



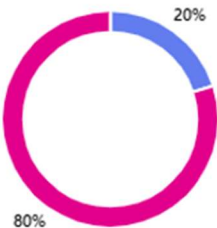
After some work on design of the game interface I sent out this survey for more stakeholder feedback from a wider range of stakeholders.

Sixth Form Friends: Responses to design survey

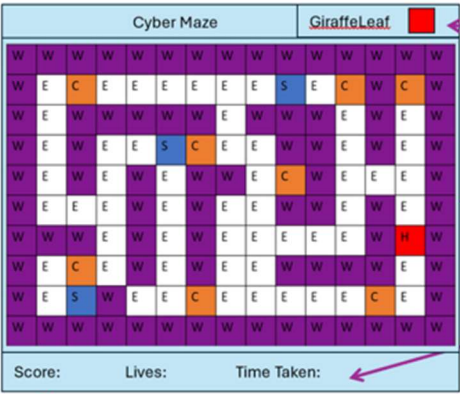


Option 1

Preference on design



As shown in the pie chart above Option 2 was widely more popular than Option 1.



Option 2

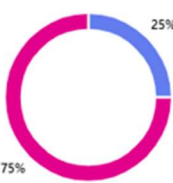
Position of Stats Bar

- Above
- Below

Position of Stats Bar

- Above
- Below

As shown having the bar below the game was preferred by 100% of the respondents that preferred option 1



As shown having the bar below the game was preferred by 75% of the respondents that preferred option 2

### **Notes from computer science teacher stakeholder meeting:**

- He would like a resource that introduces pupils to cyber security in a fun way
- He would like something that might encourage people to try for CyberFirst girls
- He would like something that might encourage more pupils to take GCSE computing
- He would like a feature that allows him to check on progress
- He would like a feature like a leaderboard
- His advice was to have different levels of challenge - e.g. easy, medium, hard

### **Notes from Cyber Security Professional Stakeholder Meeting:**

I talked to a cyber security professional today from the National Cyber Security Centre. These are the key things I noted down.

- NCSC created CyberFirst to encourage school pupils into a cyber security career.
- CyberFirst is aimed at students from all backgrounds.
- NCSC created the Year 8 CyberFirst Girls Competition to encourage female talent into cyber security.
- The aim of CyberFirst is to encourage applications for a CyberFirst bursary.
- My game proposal could be a good way of encouraging students to think cyber security is fun.
- My key aim is to make it fun so that people might be open to learning more about cyber security.

## Network manager

From:

Sent: 18 June 2024 18:33

To: (IT Manager)

Subject: Computer Science NEA

Dear Mrs A

I am hoping that you will be a stakeholder for my maze game as the Network manager if I were to run my game on the school network.

These are my ideas on what your requirements could be, but they may be wrong! Would it be possible to let me know what you think? Happy to meet or talk by email, whatever you prefer.

The School IT Network Manager will need to approve my game for use on the school network and wants to know the size of the database/file, how many players there might be, the different access permissions needed, how new accounts will be created, the requirement for backing it up, the suitability of the content for pupils, and storing pupil information correctly in accordance with GDPR. If I keep my game fairly simple, limit it to school users, and control access and data storage, this stakeholder will be happy.

Best wishes,



Mon 24/06/2024 21:03

Dear Mrs A

Thanks very much for your support. I thought the question database for the cyber clues could be saved somewhere within the school system to be accessed by the game when it is run. I was also thinking of accessing it through teams to actually run the python program. If I haven't understood your questions please do tell me! I hope I'm making sense. :)

Best wishes,

(IT Manager)



...

Wed 19/06/2024 16:26

Dear

This sounds very exciting and interesting.

Yes, I'd be happy to be involved.

My first question is how would you deploy the games?

Then would there be any external access required?

regards

Mrs A

...

I was slightly Unsure at first what Mrs A meant by the questions, but after **talking with her again and responding with what I thought she meant I gained a clearer understanding of the sort of things she needed to know in order to host my game.**

## 42. Appendix C: Source Code

### Interfaces.py

```
#interfaces.py
#code to generate the cyber maze GUI's
#and link to button event callbacks,
#including accessing the database

#Rosa
#25/10/2024

#line 126

#import python library modules
import tkinter as tk
#import tkinter.ttk as ttk      #for widgets such as combo list boxes
from tkinter import ttk
import tkinter.messagebox as mbox      #for messages
from tkinter import END      # a special char used to denote the end of a text box
string
from PIL import Image, ImageTk      #for image manipulation

##import my own files
from callbacks01 import *      #manage events

#global theme colours
global BackCol, TextCol, BtnCol
BackCol = "purple"      #default bg color
TextCol = "white"      #default font color
BtnCol = "DarkOrange2" #Button color

##event handlers

#Log in
def Submit():      #Subprogram to check Username before entering the System
    global root, Username
    #event handler for btnSubmit
    aStr = txtUsername.get()      #get entered username
    txtUsername.delete(0,END)      #remove a Username after input.
    if aStr != "Welcome please enter your username":      #stop temp text being entered
        if GetUser(aStr):      #Authentication of username
            #move guis
            frmLogIn.place(x = 5, y = 500, width = 645, height = 207)
            frmOptions.place(x = 0, y = 0, width = 645, height = 207)
            width, height = 645, 207
            root.geometry("{}x{}".format(width,height))      #reset root just in case
            #method to destroy widgets
            txtUsername.destroy()      #remove txtUsername when leaving this frame
            Username = aStr      #save Username for later use (Admin) - query for
Avatar.
            if not UserAdmin(Username):      #greying out Admin management in options
                btnAdmin["bg"] = "DarkOrange3"
```

```

        btnAdmin["fg"] = "grey80"
    else:
        btnAdmin["bg"] = BtnCol    #standard
        btnAdmin["fg"] = TextCol

    else:
        mbox.showerror("No Username Entered", "Error, This Username cannot be
submitted")    #error for temp text entered
        ##print(NumAttempts)    #test print
        return    #end procedure

def Exit():    #procedure to quit
    Cancel()
    return    #end procedure

def MoveMAccount():    #procedure for New User event
    #Set global variables
    global frmLogIn, frmManage, txtNUser, txtUsername, root, pressed

    #place entry box in
    #(may have been previously deleted for temp text reset)
    txtNUser = tk.Entry(frmManage, text = "",
                        bg = TextCol,fg = "dim gray", borderwidth=2,
font=('Bahnschrift', 11))    #settings
    txtNUser.place(x = 143, y = 68, width = 478, height = 46)    #position

    #tempory text in Entry box
    def Ntemp_text(e):
        txtNUser["fg"] = "black"
        txtNUser.delete(0,END)
        return e
    tempNreset()
    txtNUser.bind("<FocusIn>", Ntemp_text)

    #shuffle the frame placement
    frmLogIn.place(x = 5, y = 500, width = 645, height = 207)
    frmManage.place(x = 0, y = 0, width = 645, height = 207)
    width, height = 645, 207
    root.geometry("{}x{}".format(width,height))#reset the window size just in case
    #remove input box in Login GUI for reset later
    txtUsername.destroy()
    pressed = 1    #set value for the avatar selection default

def tempNreset():    #procedure to reset temp text in Manage account
    #procedure called whenever move to manage account GUI
    global txtNUser
    aStr = txtNUser.get()
    if aStr != "Please enter a username":    #check for preexisting temp text
        txtNUser.insert(0, "Please enter a username")    #if not already there enter
temp text
    return

#options
def LogIn():    #procedure for back to login
    #Declare global variables
    global txtUsername, root

```

```

    action = mbox.askyesno("Account", "If you do this you will have to log in again,
are you sure?") #Ensure the user is happy to exit this account
    if action: #is true
        Attempts = 0 #reset attempts
        resetAttempts(Attempts)
        #shuffle frames
        frmLogIn.place(x = 0, y = 0, width = 645, height = 207)
        frmOptions.place(x = 5, y = 500, width = 645, height = 207)
        width, height = 645, 207
        root.geometry("{}x{}".format(width,height)) #reset window size just in case

        #Reset the username input box - reinput temp text
        txtUsername = tk.Entry(frmLogIn, text = "",
                                bg = TextCol,fg = "dim gray", borderwidth=2,
font=('Bahnschrift', 11)) #style
        txtUsername.place(x = 143, y = 68, width = 478, height = 46) #position

        def temp_text(e): #function to reset temp text in Username input
            #change the text colour on click
            txtUsername["fg"] = "black"
            txtUsername.delete(0,END)
            return e
        tempreset() #call procedure below to reset username input box (set temp text)
        txtUsername.bind("<FocusIn>", temp_text) #set event handler

    return #end procedure

def tempreset(): #procedure to reset Username input box
    global txtUsername #globally declare button
    aStr = txtUsername.get()
    if aStr != "Welcome please enter your username": #check if temp text already input
        txtUsername.insert(0, "Welcome please enter your username") #enter temp text
    if not already there
    return #end procedure

def ManageAcc(): #procedure for event handler Manage Account button
    #Declare globals
    global frmLogIn, frmManage, txtNUser, txtUsername, root, pressed
    action = mbox.askyesno("Mannage Account", "If you do this you will have to log in
again, are you sure?") #Ensure the user is happy to exit this account
    if action: #if true
        #reset Manage account username input box
        txtNUser = tk.Entry(frmManage, text = "",
                                bg = TextCol,fg = "dim gray", borderwidth=2,
font=('Bahnschrift', 11)) #style
        txtNUser.place(x = 143, y = 68, width = 478, height = 46) #position

        #tempory text in Entry box (event handling)
        def Ntemp_text(e):
            txtNUser["fg"] = "black"
            txtNUser.delete(0,END)
            return e
        tempNreset() #reset input box
        txtNUser.bind("<FocusIn>", Ntemp_text) #reset input box event handler

    #move frames
    frmOptions.place(x = 5, y = 500, width = 645, height = 207)

```

```

        frmManage.place(x = 0, y = 0, width = 645, height = 207)
        width, height = 645, 207
        root.geometry("{}x{}".format(width,height))    #reset window size just in case
        pressed = 1    #default pressed

    return    #end procedure

def ManageAdmin():    #Admin button event handler
    if UserAdmin(Username):    #function to check if current user is admin
        #if true - move frames
        frmOptions.place(x = 5, y = 500, width = 645, height = 207)
        frmAdmin.place(x = 0, y = 0, width = 645, height = 207)
        width, height = 645, 207
        root.geometry("{}x{}".format(width,height))    #reset window size
        #input file type to be entered
        txtClue.insert(0, ".txt")
        txtMaze.insert(0, ".txt")
    else:
        #if false - access denied
        mbox.showerror("Permission Denied", "User '%s' is not permitted here"%Username)

    return    #end procedure

def Leaderboards0():    #Leaderboards button event handler
    #set globally declare variables
    global Leaderg
    #move frames
    frmOptions.place(x = 5, y = 500, width = 645, height = 207)
    frmLeader.place(x = 0, y = 0, width = 645, height = 207)
    width, height = 645, 207
    root.geometry("{}x{}".format(width,height))    #reset window size just in case

    #reload game selection options just in case
    Leaderg = Games()
    length = len(Leaderg)
    #print(length)    #test print
    count = 0
    Game = []
    for count in range (0, length):
        #add only game names to Game array
        Game.append(Leaderg[count][0])
        count = count+1
    #sorts the list alphabetically
    Game = sorted(Game)
    clbGames.config(values=Game)    #set drop down values
    return    #end procedure

def Play_Game():    #event handler for Play Game button in options
    #Declare globals
    global root, frmPlayGame, frmOptions, NickDiff, Game
    #move frames
    frmOptions.place(x = 5, y = 500, width = 645, height = 207)
    frmPlayGame.place(x = 0, y = 0, width = 645, height = 207)
    width, height = 645, 207
    root.geometry("{}x{}".format(width,height))    #reset window size just in case

    if UserAdmin(Username):    #grey out Submit button for admin users

```

```

        btnSubmitGame["bg"] = "DarkOrange3"
        btnSubmitGame["fg"] = "grey80"
    else:
        btnSubmitGame["bg"] = BtnCol      #standard style
        btnSubmitGame["fg"] = TextCol

    #reload game selection drop down just in case
    NickDiff = Games()
    length = len(NickDiff)
    #print(length)      #test print
    count = 0
    Game = []
    for count in range (0, length):
        #add game name only to Game array
        Game.append(NickDiff[count][0])
        count = count+1
    #sorts list alphabetically
    Game = sorted(Game)
    clbGameSelect.config(values=Game)      #sets array Game to drop down list

    return      #end procedure

#play game
def OptionsMove():      #event handler for back button within play game gui
    #move frames
    frmPlayGame.place(x = 5, y = 500, width = 645, height = 207)
    frmOptions.place(x = 0, y = 0, width = 645, height = 207)
    width, height = 645, 207
    root.geometry("{}x{}".format(width,height))      #reset window just in case
    clbGameSelect.set('Select a game...')      #reset temp text in drop downs
    clbGameDiff.set('Sort by Difficulty...')
    return      #end procedure

def PlaySelected():      #event handler for game submit
    SelectedGame = clbGameSelect.get()      #get game nickname for selected game
    if SelectedGame == "Select a game...":      #disallow entry of temp text
        mbox.showerror("Invalid Input", "Error, A Game must be selected")
    else:
        if not UserAdmin(Username):      #no warning message for users playing games
            PlayMSelected(SelectedGame, Username)
        else:
            mbox.showinfo("Permission Denied", "Admin user '%s' cannot play games but
can check if they load"%Username)      #warning message that Admins can't play games
            PlayMSelected(SelectedGame, Username)
    return      #end procedure

def GameDiff():      #to sort the games by difficulty
    selecteddiff = clbGameDiff.get()      #change the values in drop down
    according to filter
    Easy = []
    Med = []
    Hard = []
    All = []
    length = len(NickDiff)
    for count in range (0, length):
        #check difficulty of each game in array before saving only game name to
        specific arrays

```

```

        if NickDiff[count][1] == 1:    #if easy
            Easy.append(NickDiff[count][0])    #add only game name
##            print(Easy)    #test print
        elif NickDiff[count][1] == 2:    #if medium
            Med.append(NickDiff[count][0])
##            print(Med)    #test print
        elif NickDiff[count][1] == 3:    #if hard
            Hard.append(NickDiff[count][0])
            All.append(NickDiff[count][0])    #add all values

##            print(Hard)    #test print
    if selecteddiff == "Easy":    #sort arrays alphabetically
        Easy = sorted(Easy)
        clbGameSelect.config(values=Easy)    #set the values in the drop down
    elif selecteddiff == "Medium":
        Med = sorted(Med)
        clbGameSelect.config(values=Med)
    elif selecteddiff == "Hard":
        Hard = sorted(Hard)
        clbGameSelect.config(values=Hard)
    elif selecteddiff == "All":    #if all are selected
        All = sorted(All)
        clbGameSelect.config(values=All)
    return    #end procedure

#manage account
def which_button(button_press):    #change button graphics dependant on button pressed
    #declare globals
    global pressed

    #for previous button
    #maintain other button view
    Imagereset = ("Avatar0%i.png"%pressed)
    original_image = Image.open(Imagereset)
    # Scale the image (change width and height as needed)
    width, height = 45, 45    # Desired dimensions
    scaled_image = original_image.resize((width, height), Image.LANCZOS)
    # Convert the scaled image to a Tkinter-compatible image
    tk_image = ImageTk.PhotoImage(scaled_image)
    if pressed == 1:    #declare the image for the correct previously selected avatar
        imgAvatar1.config(image=tk_image)    #set image
        imgAvatar1.image = tk_image
    elif pressed == 2:
        imgAvatar2.config(image=tk_image)    #set image
        imgAvatar2.image = tk_image
    elif pressed == 3:
        imgAvatar3.config(image=tk_image)    #set image
        imgAvatar3.image = tk_image
    elif pressed == 4:
        imgAvatar4.config(image=tk_image)    #set image
        imgAvatar4.image = tk_image

    #change view of selected button
    imagedarker = ("Avatar%i-dark70.png"%button_press)    #change the newly selected
    button to the selected appearance
    original_image = Image.open(imagedarker)
    # Scale the image (change width and height as needed)

```

```

width, height = 45, 45 # Desired dimensions
scaled_image = original_image.resize((width, height), Image.LANCZOS)
# Convert the scaled image to a Tkinter-compatible image
tk_image = ImageTk.PhotoImage(scaled_image) #set image

if button_press== 1: #set to a specific button
    imgAvatar1.config(image=tk_image)
    imgAvatar1.image = tk_image
    pressed=1
##    print(pressed) #test print
    return 1 #return the currently selected
elif button_press == 2:
    imgAvatar2.config(image=tk_image)
    imgAvatar2.image = tk_image
    pressed=2
##    print(pressed) #test print
    return 2 #return the currently selected
elif button_press == 3:
    imgAvatar3.config(image=tk_image)
    imgAvatar3.image = tk_image
    pressed=3
    ##print(pressed) #test print
    return 3 #return the currently selected
elif button_press == 4:
    imgAvatar4.config(image=tk_image)
    imgAvatar4.image = tk_image
    pressed=4
##    print(pressed) #test print
    return 4 #return the currently selected

def Save(): #event handler for the save button within manage account
    #event handler for btnSubmit
    aStr = txtNUser.get() #get entered username
    AvatarID = pressed #number 1-4 currently selected avatar
    edit = False
    ##    print(aStr[:2]) #test print
    if UserPresence(aStr): #check if there is already a user
        #allow for accidental input of currently existing account
        edit = mbox.askyesno("Edit", "Are you sure you like to edit this account?")
        #set edit for later if statment
        #print(edit) #test print

    else:
        edit = True #edit is always true if new account

    if edit:
        action = False
        if aStr[:2] == "A_": #admin type account
            Type = "A"
            if not UserPresence(aStr): #check if wishes to create an admin account
                action = mbox.askyesno("Admin", "Are you sure you like this account to
be an Admin?")
            else:
                action = True #action is always True if editing a preexisting
account
        else:
            Type = "U"

```

```

        action = True
    if action:      #allow for editing of database data
        if aStr != "Please enter a username":    #check for temp text
            AvatarID = pressed
            AddUser(aStr, AvatarID, Type)      #data is edited within callbacks with
passed parameters
        else:
            mbox.showerror("No Username Entered", "Error, This Username cannot be
submitted")    #error due to temp text entered
        else:
            mbox.showinfo("Admin not added", "User not added, please enter a Username
without 'A_' at the start")    #error
            return    #end prodedure

def LogInN():    #event handler for button "back to Login" reset username input box
    #declare globals
    global txtUsername, txtNUser, root
    Attempts = 0    #rest attempts
    resetAttempts(Attempts)
    #move frames
    frmLogIn.place(x = 0, y = 0, width = 645, height = 207)
    frmManage.place(x = 5, y = 500, width = 645, height = 207)
    width, height = 645, 207
    root.geometry("{}x{}".format(width,height))    #reset window just in case

    txtUsername = tk.Entry(frmLogIn, text = "",
                           bg = TextCol,fg = "dim gray", borderwidth=2,
font=('Bahnschrift', 11))    #input box style
    txtUsername.place(x = 143, y = 68, width = 478, height = 46)    #position

    def temp_text(e):
        #change text colour on click
        txtUsername["fg"] = "black"
        #delete text on click
        txtUsername.delete(0,END)
        return e
    tempreset()    #reset temp text
    txtUsername.bind("<FocusIn>", temp_text)    #reset event handler for Login GUI
    txtNUser.destroy()    #delete Username input in Manage account to be reset when
needed

    return    #end procedure

#Admin Management
def OptionsA():    #event handler for back button in Admin management
    #move frames
    frmOptions.place(x = 0, y = 0, width = 645, height = 207)
    frmAdmin.place(x = 5, y = 500, width = 645, height = 207)
    width, height = 645, 207
    root.geometry("{}x{}".format(width,height))    #reset window size if needed
    txtNick.delete(0,END)    #remove Display name text when re-entering GUI.
    txtClue.delete(0,END)    #remove Clue file text when re-entering GUI.
    txtMaze.delete(0,END)    #remove Maze file text when re-entering GUI.
    clbDifficulty.set('Difficulty...')    #reset clb temp text
    clbMaxTime.set('Max Time...')    #reset clb temp text
    return    #end procedure

```

```

def AddGame():      #event handler for save button in Admin management
    Display = txtNick.get()      #get nickname
    Clue = txtClue.get()      #get clueset filename
    Maze = txtMaze.get()      #get maze filename
    Time = clbMaxTime.get()      #get max time for game
    Level = clbDifficulty.get()      #get game difficulty Level
    if Display != '' and Maze != '.txt' and Clue != '.txt' and Maze != '' and Clue !=
    '' and Time != 'Max Time...' and Level != 'Difficulty...':      #don't allow blank
    inputs
        #check 4 characters at the end of a string are the file type
        if Clue[-4:] == ".txt" and Maze[-4:] == ".txt":
            ManGame(Display, Clue, Maze, Time, Level)      #pass the parameters to this
            procedure to save the game data
        else:
            mbox.showerror("Incorrect filetype", "Error, Clue sets and Mazes must be
            in text files")      #filetype error
        else:
            mbox.showerror("Invalid Input", "Error, Not all fields have been filled please
            try again")      #error invalid input
        return      #end procedure

#Leaderboard
def ViewLeader():      #event handler for the submit button within the Leaderboards GUI
    NickName = clbGames.get()      #get the selected game nickname
    if NickName == "View specific game Leaderboard...":      #check if temp text entered
        mbox.showerror("Invalid Input", "Error, A Game must be selected")      #error
        invalid input
    else:      #valid selection
        if not UserAdmin(Username):      #User Leaderboard
            #call from function in callbacks
            Top5 = Top5Out(NickName)      #find the top 5 for specific game

            #create main Leaderboard frame
            global frmLeaderboard
            frmLeaderboard = tk.Frame(root, bg = BackCol)
            #move frames
            frmLeaderboard.place(x = 0, y = 0, width = 645, height = 207)
            frmLeader.place(x = 0, y = 500, width = 645, height = 207)

            #Title Label
            ##      print(NickName)      #test print
            Nickname = str(NickName)+ " Leaderboard"      #Title
            lblTitle = tk.Label(frmLeaderboard, text = (Nickname),
                                bg = BackCol,fg = TextCol,
                                anchor="w", font=('Bahnschrift', 16, 'bold'))      #style
            lblTitle.place(x = 25, y = 15, width = 250, height = 36)      #position

            #back Button
            btnBack = tk.Button(frmLeaderboard, text = "Back",
                                bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11)
            )
            btnBack.place(x = 561, y = 23, width = 57, height = 36)

            #global frames for the top 5
            global frmLeaderboard1, frmLeaderboard2, frmLeaderboard3, frmLeaderboard4,
            frmLeaderboard5
            frmLeaderboard1 = tk.Frame(frmLeaderboard, bg = BackCol)      #style

```

```

frmLeaderboard1.place(x = 0, y = 50, width = 560, height = 26)    #position

frmLeaderboard2 = tk.Frame(frmLeaderboard, bg = BackCol)
frmLeaderboard2.place(x = 0, y = 79, width = 560, height = 26)

frmLeaderboard3 = tk.Frame(frmLeaderboard, bg = BackCol)
frmLeaderboard3.place(x = 0, y = 108, width = 560, height = 26)

frmLeaderboard4 = tk.Frame(frmLeaderboard, bg = BackCol)
frmLeaderboard4.place(x = 0, y = 137, width = 560, height = 26)

frmLeaderboard5 = tk.Frame(frmLeaderboard, bg = BackCol)
frmLeaderboard5.place(x = 0, y = 166, width = 560, height = 26)

#results
for count in range(0,5):
##     print(Top5)    #test print
##     print(Top5[count])    #test print
    #split values into different variables for each of the top 5
    User, Score, Time, Acc = Top5[count]
    #placeholders no longer needed due to the separated variables

    First = ["User: '%s'" % User,"Score: '%s'" % Score,"Time Taken: '%s'"
% Time,"Accuracy: '%s'" % Acc]    #save all values with identical into text
    Nums = ["1. ", "2. ", "3. ", "4. ", "5. "]    #introduction numbering
    if count == 0:    #to change the frame meaning I can iterate through the
count.
        frame = (frmLeaderboard1)    #set specific frame for each
iteration
##         print(frame)    #test print
    elif count == 1:
        frame = (frmLeaderboard2)
    elif count == 2:
        frame = (frmLeaderboard3)
    elif count == 3:
        frame = (frmLeaderboard4)
    elif count == 4:
        frame = (frmLeaderboard5)

    #number label for place in top 5
    lbl = tk.Label(frame, text = (Nums[count]),
        bg = BackCol, fg = TextCol,
        anchor="w", font=('Bahnschrift', 11, 'bold'))    #style
    lbl.place(x = 11, y = 0, width = 50, height = 26)    #position

    #Username label
    label1 = tk.Label(frame, text=First[0],
        bg=BackCol, fg=TextCol, anchor="w",
        font=('Bahnschrift', 11, 'bold'))    #style
    # Place the label
    label1.place(x=61, y=0, width=150, height=26)

    #Score label
    label2 = tk.Label(frame, text=First[1],
        bg=BackCol, fg=TextCol, anchor="w",
        font=('Bahnschrift', 11, 'bold'))    #style

```

```

# Place the Label
label2.place(x=200, y=0, width=150, height=26)

#Time Label
label3 = tk.Label(frame,text=First[2],
                  bg=BackCol,fg=TextCol,anchor="w",
                  font=('Bahnschrift', 11, 'bold'))    #style
# Place the Label
label3.place(x=295, y=0, width=150, height=26)        #300 works

#Accuracy Label
label4 = tk.Label(frame,text=First[3],
                  bg=BackCol,fg=TextCol,anchor="w",
                  font=('Bahnschrift', 11, 'bold'))    #style
# Place the Label
label4.place(x=450, y=0, width=110, height=26)
count = count+1

#all labels in each column are in line.
#each iteration so this in the next frame down

#Event Handlers
btnBack['command'] = Temp_Back    #back button

clbGames.set('View specific game Leaderboard...')    #reset clb temp text
return    #end procedure
else:    #if Admin user

def UserMazeDeets():    #get the details of a selected user
    UserGame = clbUsers.get()    #get the user selected
    for count in range(0, (len(UserDetails))):    #get all the values
for the user selected
        if UserGame in UserDetails[count][0]:
            User, Score, Time, Accuracy, Easy, Medium, Hard, Complete =
UserDetails[count]
####    print(UserDetails[count] , ":)")    #test print
##    print(User, NickName, User, Score, Time, Accuracy)    #test
print

    #message box to display result
    mbox.showinfo("%s' Maze"%(NickName), '''User: '%s'\nScore:
%i'\nTime: '%s'\nAccuracy: '%s'\nAmount Easy:%i\nAmount Medium: %i\nAmount Hard:
%i\n"%s"'''%(User, Score, Time, Accuracy, Easy, Medium, Hard, Complete))
    #message box to display results
    return    #end procedure

UserDetails = LeaderA(NickName)    #get the details of all users for the
game

#frame for new gui
#global frmLeaderboard
frmLeaderboard = tk.Frame(root, bg = BackCol)
#move frames
frmLeaderboard.place(x = 0, y = 0, width = 645, height = 207)
frmLeader.place(x = 0, y = 500, width = 645, height = 207)

#Title Label
global gameselectd

```

```

gameselectd = NickName
Nickname = NickName[:len(NickName)]
##      print(Nickname)      #test print
Nickname = str(Nickname)+ " Leaderboard"      #create title for GUI
lblTitle = tk.Label(frmLeaderboard, text = (Nickname),
                    bg = BackCol,fg = TextCol,
                    anchor="w", font=('Bahnschrift', 16, 'bold'))      #style
lblTitle.place(x = 25, y = 15, width = 400, height = 36)      #position

#back Button
btnBack = tk.Button(frmLeaderboard, text = "Back",
                    bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )      #style
btnBack.place(x = 561, y = 23, width = 57, height = 36)      #position

UserD = []      #array to fill with usernames
for count in range(0, (len(UserDetails))):
    UserD.append(UserDetails[count][0])      #add only the Username

#drop down for all users
clbUsers = ttk.Combobox(frmLeaderboard, values = UserD,
                        text = "user details", state= "readonly",
                        font=('Bahnschrift', 16, 'bold'))      #style
clbUsers.set('View Details from Users last Game ...')      #temp text
clbUsers.place(x = 23, y = 65, width = 595, height = 38)      #position

#view button
btnView = tk.Button(frmLeaderboard, text = "View",
                    bg = BtnCol ,fg = TextCol,
                    font=('Bahnschrift', 11) )      #style
btnView.place(x = 464, y = 109, width = 154, height = 38)      #position

#Event Handlers
btnBack['command'] = Temp_Back      #back button
btnView['command'] = UserMazeDeets      #message box for selected user details

clbGames.set('View specific game Leaderboard...')      #reset clb temp text
return      #end procedure

def OptLeader():      #Leaderboards GUI to Options via back button
    #move frames
    frmOptions.place(x = 0, y = 0, width = 645, height = 207)
    frmLeader.place(x = 5, y = 500, width = 645, height = 207)
    width, height = 645, 207
    root.geometry("{}x{}".format(width,height))      #rest window if needed
    return      #end procedure

def Temp_Back():      #back button event handler within embedded GUI
    frmLeaderboard.destroy()      #delete the Leaderboard so that it can be updated not
    overwritten
    frmLeader.place(x = 0, y = 0, width = 645, height = 207)      #move Leaderboard to
    window position
    width, height = 645, 207
    root.geometry("{}x{}".format(width,height))      #reset window size just in case.

##procedures to create GUI's
def Main():
    Initialise()      #call procedure to create window and set values

```

```

CreateLogIn()      #create first GUI - Log In
Options()          #Options GUI
play_Game()        #game select GUI
ManageAccount()    #Manage Account GUI
Admin()            #Admin Management GUI
Leader()           #Leaderboards GUI

#make sure Avatars are correct in database.
#call function from callbacks
CheckAvatar()

root.mainloop() #continually manage GUIs until quit
return

def center_window(window):
    #ref53: https://www.geeksforgeeks.org/how-to-center-a-window-on-the-screen-in-tkinter/
    window.update_idletasks()
    width = window.winfo_width()      #identify window width
    height = window.winfo_height()     #identify window height
    screen_width = window.winfo_screenwidth() #identify full screen width
    screen_height = window.winfo_screenheight() #identify full screen height
    x = (screen_width - width) // 2    #identify X of the top left corner using maths
    y = (screen_height - height) // 2  #identify Y of the top left corner using maths
    window.geometry(f"{width}x{height}+{x}+{y}") #set window location and size
    return

def Initialise():
    #Procedure to create window container for all GUIs
    #declare Globals
    global root          #standard name for window

    root = tk.Tk() #call Library function

    root.title("Cyber Maze Quiz") #Title for window

    width, height = 645, 207
    root.geometry("{}x{}".format(width,height))
    #root.geometry("{}x{}+{}+{}".format(width,height, 0, 0)) # in the centre of the
screen
    center_window(root)
    #root.eval('tk::PlaceWindow . center')

    return #end procedure

def CreateLogIn(): #procedure to create the Log in GUI
    #declare globals
    global frmLogIn, txtUsername

    #create the frame - container
    #ref34:https://cs111.wellesley.edu/archive/cs111\_fall14/public\_html/labs/lab12/tkinter\_color.html
    #Tkinter named colours
    frmLogIn = tk.Frame(root, bg = BackCol) #style
    frmLogIn.place(x = 0, y = 0, width = 645, height = 207) #position

    #add widgets

```

```

#ref35:https://www.geeksforgeeks.org/what-are-widgets-in-tkinter/
#types of widget
#GUI Title
lblLogIn = tk.Label(frmLogIn, text = "Log In",
                    bg = BackCol,fg = TextCol,
                    anchor="w", font=('Bahnschrift', 16, 'bold')) #style
lblLogIn.place(x = 25, y = 15, width = 201, height = 36) #position

#input identifier
lblUsername = tk.Label(frmLogIn, text = "Username:",
                       bg = BackCol,fg = TextCol, anchor="w",
                       font=('Bahnschrift', 16)) #style
lblUsername.place(x = 11, y = 70, width = 124, height = 36) #position

#text input box
txtUsername = tk.Entry(frmLogIn, text = "",
                       bg = TextCol,fg = "dim gray",
                       borderwidth=2, font=('Bahnschrift', 11)) #style
txtUsername.place(x = 143, y = 68, width = 478, height = 46) #position

tempreset() #set up the temp text

#New user button
btnNewUser = tk.Button(frmLogIn, text = "New User",
                       bg = BtnCol ,fg = TextCol,
                       font=('Bahnschrift', 11) ) #style
btnNewUser.place(x = 143, y = 147, width = 130, height = 38) #position

#Submit button
btnSubmit = tk.Button(frmLogIn, text = "Log in",
                      bg = BtnCol ,fg = TextCol,
                      font=('Bahnschrift', 11) ) #style
btnSubmit.place(x = 290, y = 147, width = 130, height = 38) #position

#Cancel button
btnCancel = tk.Button(frmLogIn, text = "Cancel",
                      bg = BtnCol ,fg = TextCol,
                      font=('Bahnschrift', 11) ) #style
btnCancel.place(x = 491 , y = 147, width = 130, height = 38) #position

#set event handler

#temporary text in Entry box
#ref38:https://www.tutorialspoint.com/how-to-insert-a-temporary-text-in-a-tkinter-entry-widget

def temp_text(e):
    #change the colour on click
    txtUsername["fg"] = "black"
    #delete contents on click
    txtUsername.delete(0,END)
    return e #end function
txtUsername.bind("<FocusIn>", temp_text)

btnSubmit['command'] = Submit #connect procedure to button click
btnCancel['command'] = Exit
btnNewUser['command'] = MoveMAccount

```

```

return #end procedure

def Options():
    #options GUI
    #Declare globals
    global frmOptions, btnAdmin
    #create the frame - container
    #same size as Log on frame
    frmOptions = tk.Frame(root, bg = BackCol)    #style
    frmOptions.place(x = 5, y = 217, width = 645, height = 207)    #position

    #same position as the Log In Title
    #Option Title
    lblOptions = tk.Label(frmOptions, text = "Options",
                           bg = BackCol, fg = TextCol,
                           anchor="w", font=('Bahnschrift', 16, 'bold'))    #style
    lblOptions.place(x = 25, y = 15, width = 201, height = 36)    #position

    #input identifier
    btnManageAccount = tk.Button(frmOptions, text = "Manage Account",
                                  bg = BtnCol ,fg = TextCol,
                                  font=('Bahnschrift', 11) )    #style
    btnManageAccount.place(x = 25, y = 63, width = 285, height = 38)    #position

    #button for Login GUI
    btnLogIn = tk.Button(frmOptions, text = "Back",
                          bg = BtnCol ,fg = TextCol,
                          font=('Bahnschrift', 11) )    #style
    btnLogIn.place(x = 333, y = 63, width = 285, height = 38)    #position

    #button for Play Game GUI
    btnPlayGame = tk.Button(frmOptions, text = "Play Game",
                             bg = BtnCol ,fg = TextCol,
                             font=('Bahnschrift', 11) )    #style
    btnPlayGame.place(x = 25, y = 107, width = 285, height = 38)    #position

    #button for Admin GUI
    btnAdmin = tk.Button(frmOptions, text = "Admin Management",
                          bg = BtnCol ,fg = TextCol,
                          font=('Bahnschrift', 11) )    #style
    btnAdmin.place(x = 25, y = 151, width = 593, height = 38)    #position

    #button for Leaderboards GUI
    btnLeaderboards = tk.Button(frmOptions, text = "Leaderboards",
                                  bg = BtnCol ,fg = TextCol,
                                  font=('Bahnschrift', 11) )    #style
    btnLeaderboards.place(x = 333, y = 107, width = 285, height = 38)    #position

    #set event handler - procedures connection to buttons
    btnLogIn['command'] = LogIn
    btnManageAccount['command'] = ManageAcc
    btnPlayGame['command'] = Play_Game
    btnAdmin['command'] = ManageAdmin
    btnLeaderboards['command'] = Leaderboards0
    return #end procedure

```

```

def play_Game():
    #Play Game GUI
    #Declare Globals
    global frmPlayGame, clbGameSelect, clbGameDiff, btnSubmitGame

    #create the frame - container
    #same size as Log on frame
    frmPlayGame = tk.Frame(root, bg = BackCol)    #style
    frmPlayGame.place(x = 5, y = 429, width = 645, height = 207)    #position

    #same position as previous Title
    #Play Game Title
    lblPlayGame = tk.Label(frmPlayGame, text = "Play Game",
                            bg = BackCol,fg = TextCol,
                            anchor="w", font=('Bahnschrift', 16, 'bold'))    #style
    lblPlayGame.place(x = 25, y = 15, width = 201, height = 36)    #position

    #sort Button
    btnSort = tk.Button(frmPlayGame, text = "Sort",
                        bg = BtnCol ,fg = TextCol,
                        font=('Bahnschrift', 11) )    #style
    btnSort.place(x = 199, y = 115, width = 57, height = 36)    #position

    #back button
    btnOptions = tk.Button(frmPlayGame, text = "Back",
                           bg = BtnCol ,fg = TextCol,
                           font=('Bahnschrift', 11) )    #style
    btnOptions.place(x = 561, y = 23, width = 57, height = 36)    #position

    #enter button
    btnSubmitGame = tk.Button(frmPlayGame, text = "Submit",
                              bg = BtnCol ,fg = TextCol,
                              font=('Bahnschrift', 11) )    #style
    btnSubmitGame.place(x = 464, y = 109, width = 154, height = 38)    #position

    #combo list box - I have set some temporary values to be updated later
    tempvalues = ["These", "are", "temporary", "values"]    #values to be updated when
    GUI is accessed
    clbGameSelect = ttk.Combobox(frmPlayGame, values = tempvalues,
                                text = "Select a game", state= "readonly",
                                font=('Bahnschrift', 16, 'bold'))    #style
    clbGameSelect.set('Select a game...')    #default text
    clbGameSelect.place(x = 23, y = 65, width = 595, height = 38)    #position

    #Game Difficulty drop down
    Level = ["Easy", "Medium", "Hard", "All"]    #options
    clbGameDiff = ttk.Combobox(frmPlayGame, values = Level,
                               text = "Select Game Difficulty", state= "readonly",
                               font=('Bahnschrift', 13))    #style
    clbGameDiff.set('Sort by Difficulty...')    #default text
    clbGameDiff.place(x = 23, y = 115, width = 170, height = 36)    #position

    #event handlers - connect buttons to procedures
    btnSort['command'] = GameDiff
    btnOptions['command'] = OptionsMove
    btnSubmitGame['command'] = PlaySelected

```

```

return    #end procedure

def ManageAccount():
    #Manage account GUI
    #declare globals
    global frmManage, txtNUser, imgAvatar1, imgAvatar2, imgAvatar3, imgAvatar4

    #create the frame - container
    frmManage = tk.Frame(root, bg = BackCol)    #style
    frmManage.place(x = 655, y = 5, width = 645, height = 207)    #position

    #same position as previous Title
    #Manage Account Title
    lblManage = tk.Label(frmManage, text = "Manage Account",
                        bg = BackCol,fg = TextCol,
                        anchor="w", font=('Bahnschrift', 16, 'bold'))    #style
    lblManage.place(x = 25, y = 15, width = 201, height = 36)    #position

    #Identifier for the Entry box (same position as Login)
    lblUsername = tk.Label(frmManage, text = "Username:",
                        bg = BackCol,fg = TextCol,
                        anchor="e", font=('Bahnschrift', 16))    #style
    lblUsername.place(x = 11, y = 70, width = 122, height = 36)    #position

    #Usersme entry box (same position as Login)
    txtNUser = tk.Entry(frmManage, text = "",
                        bg = TextCol,fg = "dim gray", borderwidth=2,
                        font=('Bahnschrift', 11))    #style
    txtNUser.place(x = 143, y = 68, width = 478, height = 46)    #position

    #temporary text for input
    txtNUser.insert(0, "Please enter a username")

    #back button (for returning to Login)
    btnLogIn = tk.Button(frmManage, text = "Return to Login",
                        bg = BtnCol ,fg = TextCol,
                        font=('Bahnschrift', 11) )    #style
    btnLogIn.place(x = 464, y = 15, width = 154, height = 38)    #position

    #Identifier for the Avatar buttons
    lblAvatar = tk.Label(frmManage, text = "Avatar:",
                        bg = BackCol,fg = TextCol, anchor="e",
                        font=('Bahnschrift', 16))    #style
    lblAvatar.place(x = 40, y = 150, width = 89, height = 36)    #position

    #button to save details from manage account
    btnSave = tk.Button(frmManage, text = "Save",
                        bg = BtnCol ,fg = TextCol,
                        font=('Bahnschrift', 11) )    #style
    btnSave.place(x = 464, y = 150, width = 154, height = 38)    #position

    #1
    # Load the image using PIL
    original_image = Image.open("Avatar1-dark70.png")    #pre selected(darker image
    necessary)

    # Scale the image

```

```

width, height = 45, 45 # Desired dimensions (square)
scaled_image = original_image.resize((width, height), Image.LANCZOS)

# Convert the scaled image to a Tkinter-compatible image
tk_image = ImageTk.PhotoImage(scaled_image)

# Create a button and set the image
imgAvatar1 = tk.Button(frmManage, image=tk_image, anchor="center",
                       command=lambda imgAvatar1 = 1: which_button(imgAvatar1)) #style
imgAvatar1.place(x = 142, y = 138, width = 47, height = 47) #position

# Keep a reference to avoid garbage collection
imgAvatar1.image = tk_image

#2
#Load the image using PIL
original_image2 = Image.open("Avatar02.png") #ordinary image (with background)

# Scale the image
width, height = 45, 45 # Desired dimensions (square)
scaled_image2 = original_image2.resize((width, height), Image.LANCZOS)

# Convert the scaled image to a Tkinter-compatible image
tk_image2 = ImageTk.PhotoImage(scaled_image2)

# Create a button and set the image
imgAvatar2 = tk.Button(frmManage, image=tk_image2, anchor="center",
                       command=lambda imgAvatar2 = 2: which_button(imgAvatar2)) #style
imgAvatar2.place(x = 205, y = 138, width = 47, height = 47) #position

# Keep a reference to avoid garbage collection
imgAvatar2.image = tk_image2

#3
#Load the image using PIL
original_image3 = Image.open("Avatar03.png") #ordinary image (with background)

# Scale the image
width, height = 45, 45 # Desired dimensions (square)
scaled_image3 = original_image3.resize((width, height), Image.LANCZOS)

# Convert the scaled image to a Tkinter-compatible image
tk_image3 = ImageTk.PhotoImage(scaled_image3)

# Create a button and set the image
imgAvatar3 = tk.Button(frmManage, image=tk_image3, anchor="center",
                       command=lambda imgAvatar3=3: which_button(imgAvatar3)) #style
imgAvatar3.place(x = 264, y = 138, width = 47, height = 47) #position

# Keep a reference to avoid garbage collection
imgAvatar3.image = tk_image3

#4
#Load the image using PIL
original_image4 = Image.open("Avatar04.png") #ordinary image (with background)

```

```

# Scale the image
width, height = 45, 45 # Desired dimensions (square)
scaled_image4 = original_image4.resize((width, height), Image.LANCZOS)

# Convert the scaled image to a Tkinter-compatible image
tk_image4 = ImageTk.PhotoImage(scaled_image4)

# Create a button and set the image
imgAvatar4 = tk.Button(frmManage, image=tk_image4, anchor="center",
                        command=lambda imgAvatar4 = 4: which_button(imgAvatar4)) #style
imgAvatar4.place(x = 325, y = 138, width = 47, height = 47) #position

# Keep a reference to avoid garbage collection
imgAvatar4.image = tk_image4

#tempory text in Entry box
#ref38:https://www.tutorialspoint.com/how-to-insert-a-temporary-text-in-a-tkinter-entry-widget
def Ntemp_text(e):
    #change the colour on click
    txtNUser["fg"] = "black"
    #delete chars on click
    txtNUser.delete(0,END)
    return e
txtNUser.bind("<FocusIn>", Ntemp_text) #set click event

#Event Handlers
#send button value to function as a parameter
imgAvatar1['command'] = lambda imgAvatar1 = 1: which_button(imgAvatar1)
imgAvatar2['command'] = lambda imgAvatar2 = 2: which_button(imgAvatar2)
imgAvatar3['command'] = lambda imgAvatar3 = 3: which_button(imgAvatar3)
imgAvatar4['command'] = lambda imgAvatar4 = 4: which_button(imgAvatar4)
btnSave['command'] = Save
btnLogIn['command'] = LogInN
return #end procedure

def Admin():
    #Admin Management GUI
    #declare globals
    global frmAdmin, txtNick, txtMaze, txtClue, clbMaxTime, clbDifficulty, clbLevel

    #create the frame - same style as previous
    frmAdmin = tk.Frame(root, bg = BackCol) #style
    frmAdmin.place(x = 655, y = 220, width = 645, height = 207) #position

    #title for GUI (same position as above)
    lblAdmin = tk.Label(frmAdmin, text = "Admin Management",
                        bg = BackCol,fg = TextCol,
                        anchor="w", font=('Bahnschrift', 16, 'bold')) #style
    lblAdmin.place(x = 25, y = 15, width = 201, height = 36) #position

    #identifier for maze nickname input box
    lblNick = tk.Label(frmAdmin, text = "Display Name:",
                        bg = BackCol,fg = TextCol,
                        anchor="e", font=('Bahnschrift', 12)) #style

```

```

lblNick.place(x = 24, y = 63, width = 136, height = 31)      #location

#input box for maze nickname
txtNick = tk.Entry(frmAdmin, text = "",
                    bg = TextCol,fg = "black", borderwidth=2,
                    font=('Bahnschrift', 12))      #style
txtNick.place(x = 170, y = 67, width = 300, height = 28)      #position

#identifier for clueset file input box
lblClue = tk.Label(frmAdmin, text = "Clue Set filename:",
                   bg = BackCol,fg = TextCol,
                   anchor="e", font=('Bahnschrift', 12))      #style
lblClue.place(x = 24, y = 102, width = 136, height = 31)      #position

#input box for Clue set filename
txtClue = tk.Entry(frmAdmin, text = "",
                   bg = TextCol,fg = "black", borderwidth=2,
                   font=('Bahnschrift', 12))      #style
txtClue.place(x = 170, y = 105, width = 300, height = 28)      #position

#identifier for Maze filename input box
lblMaze = tk.Label(frmAdmin, text = "Maze filename:",
                   bg = BackCol,fg = TextCol,
                   anchor="e", font=('Bahnschrift', 12))      #style
lblMaze.place(x = 24, y = 139, width = 136, height = 31)      #position

#input box for Maze filename
txtMaze = tk.Entry(frmAdmin, text = "",
                   bg = TextCol,fg = "black", borderwidth=2,
                   font=('Bahnschrift', 12))      #style
txtMaze.place(x = 170, y = 143, width = 300, height = 28)      #position

#button to save Maze details to database
btnSave = tk.Button(frmAdmin, text = "Save",
                    bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )      #style
btnSave.place(x = 483, y = 143, width = 135, height = 28)      #position

#max time drop down
clbTimeValues = ["00:02:00"], ["00:05:00"], ["00:10:00"], ["00:15:00"],
["00:20:00"], ["00:25:00"], ["00:30:00"]      #max time options
clbMaxTime = ttk.Combobox(frmAdmin, values = clbTimeValues, state= "readonly",
                          font=('Bahnschrift', 11, 'bold'))      #style
clbMaxTime.set('Max Time...')      #tempory text in drop down
clbMaxTime.place(x = 483, y = 67, width = 135, height = 28)      #position

#Level selection drop down
clbLevel = ["Easy"],["Medium"],["Hard"]      #difficulty level options
clbDifficulty = ttk.Combobox(frmAdmin, values = clbLevel, state= "readonly",
                             font=('Bahnschrift', 11, 'bold'))      #style
clbDifficulty.set('Difficulty...')      #temporary text in drop down
clbDifficulty.place(x = 483, y = 105, width = 135, height = 28)      #position

#back button (same style as previous GUIs)
btnOptions = tk.Button(frmAdmin, text = "Back",
                       bg = BtnCol ,fg = TextCol,
                       font=('Bahnschrift', 11) )      #style
btnOptions.place(x = 561, y = 23, width = 57, height = 36)      #position

```

```

#Event Handlers - 2 buttons
btnOptions['command']= OptionsA
btnSave['command']= AddGame

return    #end procedure

def Leader():
#Leaderboard selection GUI
#declare globals
global frmLeader, Leaderg, clbGames

#frame - container for widgets (same size and style as previous
frmLeader = tk.Frame(root, bg = BackCol)    #style
frmLeader.place(x = 655, y = 429, width = 645, height = 207)    #position

#Leaderboards GUI Title
#same position as previous titles
lblLeader = tk.Label(frmLeader, text = "Leaderboards",
                    bg = BackCol,fg = TextCol,
                    anchor="w", font=('Bahnschrift', 16, 'bold'))    #style
lblLeader.place(x = 25, y = 15, width = 201, height = 36)    #position

#submit button to view a specific game leaderboard
btnView = tk.Button(frmLeader, text = "View",
                    bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )    #style
btnView.place(x = 464, y = 109, width = 154, height = 38)    #position

#back button to navigate to options GUI
btnBack = tk.Button(frmLeader, text = "Back",
                    bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )    #style
btnBack.place(x = 561, y = 23, width = 57, height = 36)    #position

#selection box for game leaderboard
Leaderg=["Temporary"],["Leaderboard"], ["Values"]    #to allow for updating
whenever the GUI is accessed
clbGames = ttk.Combobox(frmLeader, values = Leaderg,
                    text = "View specific game leaderboards", state= "readonly",
                    font=('Bahnschrift', 16, 'bold'))    #style
clbGames.set('View specific game Leaderboard...')    #drop down temporary text
clbGames.place(x = 23, y = 65, width = 595, height = 38)    #position

#event handlers - 2 buttons
btnView['command'] = ViewLeader
btnBack['command'] = OptLeader
return    #end procedure

##Main
Main()    #call main to load in GUIs

```

## Callbacks01.py

```
#callbacks01.py
#Library of subprograms to support the GUIs
#Rosa 02/11/24

import tkinter.messagebox as mbox          #for messages
from PlayGame01 import *                  #access game play code library
from LogOnDataBase import *               #access the database code library

#Log in
def GetUser(aStr):
    #function to validate and authenticate a username
    #print(aStr + ":", end = '')
    C_MaxAttempts = 3 #const - max number of attempts allowed
    global NumAttempts    #allows for modification of NumAttempts
    AuthLogIn = False     #default

    if IsValid(aStr) and IsAuth(aStr):      #to check if aStr (username) is valid and
authentic
##        print("True") #test print
        AuthLogIn=True #enter if statement below
    else:  #failed validation or authentication
##        print("False") #test print
        NumAttempts = NumAttempts + 1    #increment global NumAttempts count
##        print("%i Login Attempt(s)" %NumAttempts) #test print

    #skip unless authorised input
    if AuthLogIn:      #successful attempt
        #get users details
        aQry = '''SELECT * FROM User
                WHERE UserName = "%s"''' %aStr    #query to find user details as a
double check
        AnsTable = RunQuery(DBFile,aQry)    #is auth proved they are there
##        print(AnsTable) #test print
        #print("Welcome '%s'" %AnsTable[0][1])
        mbox.showinfo("Log In Success", "Welcome '%s'" %AnsTable[0][1])    #use
Username from
        return True    #end function return true

    elif NumAttempts == C_MaxAttempts: #failed log in after max goes
##        print("Log In failed after %i goes" %NumAttempts) #test print
        mbox.showerror("FAILED", "Log In failed after %i attempts" %NumAttempts)
#message box for 3 failed attempts
        quit()    #quit the game
        return False    #end function return false

def Cancel():    #to quit the game
    action = mbox.askyesno("Cancel", "Are you sure?, This will close down this
application")    #verification required
    if action:    #ask before shutting down
        quit()    #quit the game
    return    #end procedure
```

```

def resetAttempts(Attempts):    #attempt manager
    global NumAttempts
    NumAttempts = Attempts #reset attempts for re-entry into portal
    return #end procedure

#manage account centered sub programs
def AddUser(aStr, AvatarID, Type):
    #function to validate and authenticate a username
    ## print(aStr + ":", end = '') #test print
    Valid = False #default

    if IsValid(aStr): #validate input username
        ## print("Username is valid") #test print
        Valid = True #valid if the Username is
    ## else:
    ## print("not valid") #test print
    if Valid:
        if NotAuth(aStr): #check if the username is authentic
            #if new user:
            aDML = '''INSERT INTO User(Username,Type, AvatarID) VALUES("%s", "%s",
%i)''' % (aStr, Type, AvatarID) #using SQL to add a record to the User table
            AddData(DBFile, aDML)
            aQry = '''SELECT * FROM User
                WHERE UserName = "%s"''' %aStr #double check
            AnsTable = RunQuery(DBFile,aQry) #is auth proved they are there
            ## print(AnsTable) #test print
            if AnsTable[0][2] == "U": #get the user type for message box
                Type = "User"
            else:
                Type = "Admin"
            mbox.showinfo("Account Created", "Welcome '%s' '%s'" %(Type,
AnsTable[0][1])) #personalised User added pop up message
            else:
                #editing a current user:
                #get users details
                aQry = '''SELECT * FROM User
                    WHERE UserName = "%s"''' %aStr
                AnsTable = RunQuery(DBFile,aQry) #is auth proved they are there
                #print(AnsTable) #Test print (double check record is there)
                aDML = ('''UPDATE User SET AvatarID=%i WHERE
UserName="%s"'''%(AvatarID, aStr)) #DML update to edit the current user input
                AddData(DBFile, aDML) #execute DML
                #get users details to test
                aQry = '''SELECT * FROM User
                    WHERE UserName = "%s"''' %aStr
                AnsTable = RunQuery(DBFile,aQry)
                ## print(AnsTable) #test print
                if AnsTable[0][2] == "U": #get the user type for message box
                    Type = "User"
                else:
                    Type = "Admin"

                mbox.showinfo("Account Edited", "'%s' '%s' has been successfully
edited" % (Type, AnsTable[0][1])) #personalised User edited pop up message
    return #end proceure

```

```

def NotAuth(aStr):
    #function to query the User table in the database to a match to a valid aStr
    #without error pop ups (to use in the Manage account GUI)

    #create query - to find User record (or not)
    Qry = '''SELECT UserName FROM User
            WHERE UserName = "%s"''' %aStr
    #run query - DBFile set global in LogOnDatabase for Database file
    AnsTable = RunQuery(DBFile, Qry)

    #test result - Usernames are unique
    if AnsTable:    #found match
        return False    #end function return False
    else:
        return True    #end function return True

def IsAuth(aStr):
    #function to query the User table in the database to a match to a valid aStr
    #with error messages (to use in the Login GUI)

    #create query - to find User record (or not)
    Qry = '''SELECT UserName FROM User
            WHERE UserName = "%s"''' %aStr
    #run query - DBFile set global in LogOnDatabase for Database file
    AnsTable = RunQuery(DBFile, Qry)

    #test result - Usernames are unique
    if AnsTable:    #found match
        return True    #end function return True (username found)
    else:
        mbox.showerror("Authentication Failed", "Error Authentication failed,
        '%s' is not a valid Username"%aStr)    #error message for non existing username
        return False    #end function return false (username not found)

def IsValid(aStr):
    #function to perform a validation checks on aStr
    #Return True if ALL passed false otherwise
    #used for both Login and Manage Account
    ValidChars = "abcdefghijklmnopqrstuvwxyz0123456789_"    #string = array of
    chars available

    #use imbedded if statements for each check with return True at the inner most
    if
    if aStr != "":    #passes validation check - NULL

        if len(aStr) >=1 and len(aStr) <= 12:    #passes length check - 1-12 inclusive

            #conditional linear search to check char by char
            IsGood = True    #boolean loop control valid
            CharNum = 0    #index into a Str

            while IsGood and CharNum < len(aStr):    #to check for invalid characters
                aChar = aStr[CharNum]    #get a char to check - loop through all
                for while

                    if aChar.lower() in ValidChars:    #good char
                        CharNum = CharNum + 1    #get next char

```

```

        else:
            IsGood = False #quit checking
            #why did Loop quit?
            if IsGood:
                return True #aStr passes all validation checks - return True
            else:
                mbox.showerror("Validation Failed", "Error Validation failed,
Username: '%s' contains forbidden characters"%aStr) #custom error message for
specific validation error
                return False #aStr failed type check - invalid char(s)

        else: #fails length check
            mbox.showerror("Validation Failed", "Error Validation failed, Username:
'%s' is too long"%aStr) #custom error message for specific validation error
            return False #aStr failed length check - too long >12
        else: #fails presence check
            mbox.showerror("Validation Failed", "Error Validation failed, No Username
entered") #custom error message for specific validation error
            return False #aStr failed presence check - NULL

#play game - drop down
def Games():
    #create query - to find the Nickname and difficulty for all available games.
    Qry = '''SELECT NickName,Difficulty FROM Maze'''
    #run query - DBFile set global in LogOnDatabase for Database File
    AnsTable = RunQuery(DBFile, Qry)
    ## print(AnsTable) #test print
    return AnsTable #end function return List found in Query

#Admin Management
def ManGame(NickName, CFilename, MFilename, MaxTime, Difficulty):
    #function to validate and authenticate a username
    ## print(NickName + ":", end = '')
    ## print(CFilename + ":", end = '')
    ## print(MFilename + ":", end = '')
    Valid = False #default value for Valid

    if Difficulty == "Hard": #to get difficulty into the correct format
        Difficulty = 3
    elif Difficulty == "Medium":
        Difficulty = 2
    else:
        Difficulty = 1

    #to stop SQL injection - check for Valid inputs for all input boxes
    if ValidInp(CFilename):
    ## print("Clueset is valid") #test print
        if ValidInp(MFilename):
    ## print("MFilename is valid") #test print
            if ValidInp(NickName):
    ## print("NickName is valid") #test print
                Valid = True #only true if all the above inputs are true

    if Valid:
        if InMaze(NickName): #check if the Game already exists (by NickName)

```

```

        action = mbox.askyesno("Existing Game", "This game already exists,
Would you like to edit this game?")    #if accidentally entered a current game it
will be caught and brought to attention

        if action:    #if game is to be edited
            Table = AddClues(CFilename)    #subprogram to add Clue set record
if necessary - returns ClueID
            aDML = ('''UPDATE Maze SET MazeFilename="%s",
                        Difficulty = %i,
                        MaxTime = "%s",
                        ClueID = %i
                        WHERE NickName="%s"'''%(MFilename, Difficulty, MaxTime, Table,
NickName))    #DML statement to update fields where the nickname is as stated
            AddData(DBFile, aDML)
            #get Maze details to test
            aQry = '''SELECT * FROM Maze
                        WHERE NickName = "%s"''' %NickName
            AnsTable = RunQuery(DBFile,aQry)
            ## print(AnsTable) #test print
            mbox.showinfo("Game Edited", "Game '%s' has been edited"
%AnsTable[0][5])    #custom success message
        else:
            #if the game does not want to be edited
            mbox.showerror("Game in Use", "Please choose another Display name,
's' already in use." %NickName)    #custom error message - Asmin must choose
different NickName(Display Name)

        else:    #if game is not currently in the table
            Table = AddClues(CFilename)    #subprogram to add Clueset record if
necessary - returns ClueID

            aDML = '''INSERT INTO Maze(MazeFilename, Difficulty, MaxTime, ClueID,
NickName) VALUES("%s", %i, "%s", %i, "%s")''' % (MFilename, Difficulty, MaxTime,
Table, NickName)    #DML Statement to add Maze record to Maze Table
            AddData(DBFile, aDML)
            #test if DML was successful
            aQry = '''SELECT * FROM Maze
                        WHERE NickName = "%s"''' %NickName
            AnsTable = RunQuery(DBFile,aQry)    #get maze details
            ## print(AnsTable) #test print
            mbox.showinfo("Game Created", "Game '%s' Successfully Created"
%AnsTable[0][5])    #custom success message for game creation

        return    #end procedure

def AddClues(CFilename):
    #function to add Clueset record if necessary when creating a game
    if not InClue(CFilename):    #new clueset
        aDML= '''INSERT INTO ClueSet(ClueFilename) VALUES("%s")''' %CFilename
        #DML statement to add new record to Clueset Table
        AddData(DBFile, aDML)

        #test
        aQry = '''SELECT * FROM ClueSet
                    WHERE ClueFilename = "%s"''' %CFilename    #check for Clueset
filename
        AnsTable= RunQuery(DBFile,aQry)

```

```

        mbox.showinfo("Clue Set Added", "Clue Set '%s' Successfully Added"
%AnsTable[0][1])      #custom success message

        aQry = '''SELECT ClueID FROM ClueSet
                WHERE ClueFilename = "%s"''' %CFilename      #find ClueID to return
to ManGame function
        AnsTable= RunQuery(DBFile,aQry)
        AnsTable = AnsTable[0][0]
        return AnsTable      #end function return ClueID

def ValidInp(aStr):
    #validate for Admin Management
    ValidChars = "abcdefghijklmnopqrstuvwxyz0123456789._-"      #space not allowed
to prevent SQL injection attacks

    #must pass all validation checks to return True
    if aStr != "": #passes presence check

        if len(aStr) >=5 and len(aStr) <= 20: #passes length check - 5-20 inclusive

            #conditional linear search to check char by char
            IsGood = True      #boolean loop control valid
            CharNum = 0      #index into a Str

            while IsGood and CharNum < len(aStr):      #to check for valid characters
                aChar = aStr[CharNum]      #get a char to check

                if aChar.lower() in ValidChars:      #good char
                    CharNum = CharNum + 1      #get next char
                else:
                    IsGood = False      #quit checking
                    #why did loop quit?
                    if IsGood:
                        return True      #aStr passes all validation checks - return True
                    else:
                        mbox.showerror("Validation Failed", "Error Validation failed, Field
entered '%s' contains forbidden characters"%aStr)      #custom error message for
specific validation error
                        return False      #aStr failed type check - invalid char(s)

            else: #fails length check
                mbox.showerror("Validation Failed", "Error Validation failed, Field
entered '%s' must be between 5 and 20 chars"%aStr)      #custom error message for
specific validation error
                return False      #aStr failed length check - (<5 or >20)
            else: #fails presence check
                mbox.showerror("Validation Failed", "Error Validation failed, an input
is required")      #custom error message for specific validation error
                return False      #aStr failed presence check - NULL

def InMaze(Name):
    #function to query the Maze table in the database to check if the Maze is
already in the database

    #create query - find related Maze record for the NickName entered (or not)
    Qry = '''SELECT * FROM Maze
            WHERE NickName = "%s"''' %Name

```

```

#run query - DBFile set global in LogOnDatabase for Database file
AnsTable = RunQuery(DBFile, Qry)

#check for presence - match or no match
if AnsTable:    #found match
    return True    #end function return True (Maze record found)
else:
    return False    #end function return False - no match

def InClue(Name):
#function to query the Clueset table in the database to check if Clueset is already
in the database

    #create query - find related ClueSet record for the Clue filename (or not)
    Qry = '''SELECT ClueID FROM ClueSet
            WHERE ClueFilename = "%s"''' %Name
    #run query - DBFile set global in LogOnDatabase for Database file
    AnsTable = RunQuery(DBFile, Qry)

    #check for presence - match or no match
    if AnsTable:    #found match
        return True    #end function return True - (Clueset record found)
    else:
        return False    #end function return False - no match

def UserAdmin(User):
#function to check if User account type = Admin
#create query
Qry = '''SELECT * FROM User
        WHERE UserName = "%s"''' %User
#run query - DBFile set global in LogOnDatabase for Database file
AnsTable = RunQuery(DBFile, Qry)
#print(AnsTable)    #test print

#check for account type - return True if admin
if AnsTable[0][2] == "A":
    return True    #end function return True - (Admin Account)
else:
    return False    #end function return False - (User Account)

#Leaderboards
def FindMaze(aStr):
#function to find User_Maze records for a specific game

#create first query - find MazeID for aStr(NickName)
Qry = '''SELECT MazeID FROM Maze
        WHERE NickName = "%s"''' %aStr
MazeID = RunQuery(DBFile, Qry)
##    print(MazeID) #test print
##    print(MazeID[0][0]) #test print

#create second query - find all User_Maze records for the MazeID found above
Qry = '''SELECT * FROM User_Maze
        WHERE MazeID = %i''' %MazeID[0][0]
UserMaze = RunQuery(DBFile, Qry)
##    print(UserMaze) #test print
return UserMaze    #return full list of User results for the specific game

```

```

def FindUser(User):
    #function to return the Username associated with the entered UserID
    aQry = '''SELECT UserName FROM User
              WHERE UserID = %i'''%User
    Username = RunQuery(DBFile, aQry)
    Username = Username[0][0]
    return Username    #end function and return associated Username

#User Leaderboard
def Top5Out(UserMaze):
    #function to find the top 5 results for a specific game, order them and return
    the list
    Nickname = UserMaze
    #using MazeID to collate all User maze scores for specific mazes
    UserMazeorder = FindMaze(Nickname)
    ##    print(UserMazeorder) #test print
    #remove incompleted games
    if UserMazeorder != []:    #don't run loop for null values (avoid errors)
        for count in range(0, len(UserMazeorder)):    #loop to remove games
            ##        print(count) #test print
            if int(UserMazeorder[count][9]) == 0:    #not complete games
                UserMazeorder.pop(count)    #remove from the list

    #Order these scores into the top 5
    #order by Score(with time as a back up) - (Learned using ref 51+52)
    UserMazeorder =sorted(UserMazeorder, key = lambda x: (-x[1], x[2]))

    count = 0    #count index for loop
    Top5 = []    #empty array to add only top 5 values
    for count in range(0, 5):
        if count<len(UserMazeorder):    #may be less than 5 records for a game
            User = UserMazeorder[count][7]
            User = FindUser(User)    #find Username for related UserID
            Acc = UserMazeorder[count][6]    #accuracy recorded
            #top5 - Username, Score, TimeTaken, Accuracy
            Top5.append((User, str(UserMazeorder[count][1]),
UserMazeorder[count][2], Acc))    #only add values that are to be displayed
        else:    #games with less that 5 records need to have placeholders
            blank = (" ")    #NULL Value
            Top5.append((blank, blank, blank, blank))    #add to end of list as it is
            count = count + 1    #increment count
    #print(Top5)    #test print
    return Top5    #end function return array of top 5

#Admin Leaderboard
def LeaderA(UserMaze):
    #function to get results and sort them for the Admin Leaderboard
    Nickname = UserMaze
    #using Maze ID to collate all User maze scores for specific mazes
    UserMazeorder = FindMaze(Nickname)
    #Order these scores like the top 5 for the User Leaderboard.
    #order by Score
    UserMazeorder.sort(key=lambda test_list: test_list[1], reverse=True)
    count = 0    #index for loop
    UserDetails = []
    for count in range(0, (len(UserMazeorder))):
        #Find all values that will be within the Users score pop up messages

```

```

        User = UserMazeorder[count][7]
        User = FindUser(User)      #find Username using UserID
        Scr = UserMazeorder[count][1]    #score
        TT = UserMazeorder[count][2]    #TimeTaken
        Acc = UserMazeorder[count][6]    #Accuracy
        Easy = UserMazeorder[count][3]    #Number Easy
        Med = UserMazeorder[count][4]    #Number Medium
        Hard = UserMazeorder[count][5]    #Number Hard
        Completed = UserMazeorder[count][9]    #boolean number associated with
completion
        if Completed == 0:      #0=False, 1=True
            Completed = "Game not Completed"
        else:
            Completed = "Game Completed"
        UserDetails.append((User, Scr, TT, Acc, Easy, Med, Hard, Completed))
        count = count + 1    #increment count
## print(UserDetails) #test print

#sort UserDetails by completed (alphabetically(c before n)) then reverse score
UserDetails =sorted(UserDetails, key = lambda x: (x[7],-x[1]))

    return UserDetails    #end function return ordered UserDetails List

#play game GUI
def PlayMSelected(Nickname, Username):
    #function to set up and manage the stats from playing a Maze.

    #need to Query the database for User and Maze Record.

    #create query - User Record
    Qry = '''SELECT * FROM User
           WHERE UserName = "%s"'''%Username
    UserRecord = RunQuery(DBFile, Qry)
    UserRecord = UserRecord[0]    #get tuple only
## print(UserRecord) #test print

    #create query - Avatar Filename
    Qry = '''SELECT AvatarFilename FROM Avatar
           WHERE AvatarID = "%s"'''%UserRecord[3]
    AvatarFilename = RunQuery(DBFile, Qry)[0][0]    #get tuple only
## print(AvatarFilename) #test print

    #create query - MazeRecord
    Qry = '''SELECT * FROM Maze
           WHERE NickName = "%s"'''%Nickname
    MazeRecord = RunQuery(DBFile, Qry)[0]    #get tuple only
## print(MazeRecord) #test print

    #crate query - get Clue file
    aQry = '''SELECT ClueFilename FROM ClueSet
            WHERE ClueID = "%s"''' %MazeRecord[4]
    Clue = RunQuery(DBFile,aQry)[0][0]
## print(Clue) #test print

    #validate the files - Maze file, Clueset file
    if ValidFile(MazeRecord, Clue, UserRecord, AvatarFilename):    #files are Valid
        if not UserAdmin(Username):    #Account is User

```

```

        #play game and save stats in individual variables to be saved into the
        database.
        Score, Lives, TimeTaken, NumEasy, NumMed, NumHard, Accuracy, Comp =
        PlayGame(MazeRecord, Clue, UserRecord, AvatarFilename)    #play game
        Save = Save_to_DB(Score, Lives, TimeTaken, NumEasy, NumMed, NumHard,
        Accuracy, Comp, Username, Nickname)    #save to database - use variables saved above
        if Save:    #correct pop up message for if saved is true or false
            mbox.showinfo("Added", "Data correctly added to database")
#verification for the User
        else:
            mbox.showerror("Error", "There was an error saving your game
            details to the database")    #for testing(should not get this)
        else:    #Account is Admin - cannot play games
            #check if file validation works
            mbox.showinfo("Loaded", "Game has Loaded correctly")
            mbox.showerror("Permission Denied", "Admin user '%s' is not permitted
            to play games"%Username)    #error message for Admin playing games

def ValidFile(MazeRecord, Clue, UserRecord, AvatarFilename):
    #to check if files are valid before running the game
    aFileM = MazeRecord[1]    #Maze filename
    aFileC = Clue    #Clueset filename
    Presence = LoadFile(aFileC)    #Use LoadFile to load in Clueset file

    if Presence != [] and (len(Presence)>8):    #check for NULL and >8
        Presence = LoadFile(aFileM)    #Use LoadFile to load in Maze file

    if (Presence[0] == (len(Presence[0])*"W")):    #wall surrounds Maze
        if len(Presence[0])>9:    #at least a width of 10

            #iterate to remove accepted Chars (W,E,C,S,H)
            for row in range(0, len(Presence)):
                #replace accepted values with NULL
                Presence[row] = ((str(Presence[row])).upper()).replace("W", "")
                Presence[row] = ((str(Presence[row])).upper()).replace("E", "")
                Presence[row] = ((str(Presence[row])).upper()).replace("C", "")
                Presence[row] = ((str(Presence[row])).upper()).replace("S", "")
                Presence[row] = ((str(Presence[row])).upper()).replace("H", "")
            ##
            print(Presence[row])    #test print
            ##
            print("row:",row)    #test print

            #ref47 remove empty from array
            #https://stackoverflow.com/questions/3845423/remove-empty-strings-
            from-a-list-of-strings
            while '' in Presence:
                Presence.remove('')    #remove empty

            ##
            print(Presence)    #test print

            if Presence == []:    #maze file is valid
                return True    #end function all files are valid - return True
            else:
                ##
                print(Presence[0])    #test print
                mbox.showerror("File loading Failed", "Error loading file '%s',
                Game contains forbidden characters"%aFileM)    #custom message for specific error

        else:

```

```

        mbox.showerror("Insuficient rows", "Game must have at least 10
columns")    #custom message for specific error
    else:
        mbox.showerror("File loading Failed", "Error loading file '%s'"%aFileM)
#custom message for specific error

    return False    #end function error has occurred - return False

def Save_to_DB(Score, Lives, TimeTaken, NumEasy, NumMed, NumHard, Accuracy, Comp,
Username, Nickname):
    #Function to Save Game stats for game to database in the User_Maze field

##    print(Score, Lives, TimeTaken, NumEasy, NumMed, NumHard, Accuracy, int(Comp),
Username, Nickname) #test print

    #create query - find UserID for Username
    aQry = '''SELECT UserID From User WHERE UserName = "%s"'''%(Username)
    #run query - DBFile global in LogOnDatabase
    UserID = RunQuery(DBFile, aQry)
    UserID = UserID[0][0]    #find the User ID for the named record
##    print(UserID)    #test

    #create query - find MazeID for Game NickName
    aQry = '''SELECT MazeID From Maze WHERE NickName = "%s"'''%(Nickname)
    #run query - DBFile global in LogOnDatabase
    MazeID = RunQuery(DBFile, aQry)
    MazeID = MazeID[0][0]    #find the Maze ID for the named record
##    print(MazeID)    #test

    #create query - find User_Maze record for UserID and MazeID (or not)
    aQry = '''SELECT * From User_Maze WHERE MazeID = %i AND UserID = %i'''%(MazeID,
UserID)
    GameUsers = RunQuery(DBFile, aQry)
##    print(GameUsers) #test print

    if not GameUsers:    #is there already a record under these details?
        #DML statement to add a record to User_Maze
        aDML= '''INSERT INTO User_Maze(Score, TimeTaken, NumEasy, NumMedium,
NumHard, Accuracy, UserID, MazeID, COMP)
VALUES(%i, "%s", %i, %i, %i, "%s", %i, %i, %d)''' %(Score,
TimeTaken, NumEasy, NumMed, NumHard, Accuracy, UserID, MazeID, Comp)
        #execute DML
        AddData(DBFile, aDML)    #new record
##        print("UserMaze Added") #test print

    else:
        #there is a record under these details
        if len(GameUsers) == 1:    #there is a record under these details
            #DML Statement to update a User_Maze record when the UserID and MazeID
are as above.
            aDML= '''UPDATE User_Maze SET Score = %i, TimeTaken = "%s", NumEasy =
%i, NumMedium = %i, NumHard = %i, Accuracy = '%s', UserID = %i, MazeID = %i, COMP =
%d WHERE MazeID = '%s' AND UserID = '%s' ''' %(Score, TimeTaken, NumEasy, NumMed,
NumHard, Accuracy, UserID, MazeID, Comp, MazeID, UserID)
            AddData(DBFile, aDML)    #append record
##            print("Update UserMaze Added") #test print

```

```

        else:      #there is more than 1 record for a user
                    #this should not be possible - precaution to reduce errors
                    aDML= '''DELETE FROM User_Maze WHERE MazeID = '%s' AND UserID = '%s' '''
%(MazeID, UserID)#DML statement to delete all records for these conditions (if any)
                    #execute DML - DBFile global in LogOnDatabase
                    AddData(DBFile, aDML)

                    #DML statement to insert a new record into the User_Maze Table
                    aDML= '''INSERT INTO User_Maze(Score, TimeTaken, NumEasy, NumMedium,
NumHard, Accuracy, UserID, MazeID, COMP) VALUES(%i, "%s", %i, %i, %i, "%s", %i, %i,
%d)''' %(Score, TimeTaken, NumEasy, NumMed, NumHard, Accuracy, UserID, MazeID,
Comp)

                    #DBFile global in LogOnDatabase
                    AddData(DBFile, aDML)          #new record

        #test for successful saving
        #create query
        aQry = '''SELECT Score, TimeTaken, NumEasy, NumMedium, NumHard, Accuracy,
UserID, MazeID, COMP From User_Maze WHERE MazeID = %i AND UserID = %i'''%(MazeID,
UserID)
        #run query - DBFile global in LogOnDatabase
        GameUsers = RunQuery(DBFile, aQry)
        ##      print(GameUsers)      #check if successful
        ##      print((Score, TimeTaken, NumEasy, NumMed, NumHard, Accuracy, UserID, MazeID,
int(Comp)))

        #check(not necessary at end due to testing) - won't catch duplicates
        if GameUsers[0] == (Score, TimeTaken, NumEasy, NumMed, NumHard, Accuracy,
UserID, MazeID, int(Comp)):
            return True      #end function saved correctly to database - return True
        else:
            return False      #end function error saving to database - return False

def UserPresence(User): #to check for a username
    #create query - find record for Username
    aQry = '''SELECT * FROM User
            WHERE Username = "%s"'''%User
    #run query - DBFile global in LogOnDatabase
    Username = RunQuery(DBFile, aQry)
    return Username      #end function return User record

def CheckAvatar():
    #function to ensure the Avatars are correct when game system is run
    for check in range(0, 4):
        ID = str(check + 1)
        AvatarFilename = "Avatar" + ID + ".png"      #concatonate the correct
filename
        #create query - check if Avatar filename is in the database
        #(different each loop)
        aQry = '''SELECT * From Avatar WHERE AvatarID = %i'''%(check+1)
        #run query - DBFile global in LogOnDatabase
        AnsTable = RunQuery(DBFile, aQry)
        ##      print(AnsTable)      #test print

        if not AnsTable:      #is there a record?
            #if no:

```

```

aDML= '''INSERT INTO Avatar(AvatarID, AvatarFilename) VALUES(%i,
"%s")''' %((check+1), AvatarFilename)  #DML statement to add a new record
AddData(DBFile, aDML)
##      print("Avatar Added")  #test print
##      print(AvatarFilename)  #test print
      elif AnsTable[0][1] != AvatarFilename:      #change the Avatar filename
aDML = ('''UPDATE Avatar SET AvatarFilename="%s" WHERE AvatarID=
%i'''%(AvatarFilename, (check+1)))  #DML statement to update Avatarfilename
      #execute DML
      AddData(DBFile, aDML)
##      print("Avatar Edited")  #test print
      #else:
##      print("The Avatars are correct")  #the avatar record is correct

      #create query - test query to view all data in the Avatar Table
aQry = '''SELECT * From Avatar'''
      #run query
AnsTable = RunQuery(DBFile, aQry)
##      print(AnsTable)  #display records for testing
      return  #end procedure

##MAIN##
#run when the file is first opened
resetAttempts(0)  #reset the number of Attempts to 0
CheckAvatar()  #reset Avatars (always correct)

```

## LogOnDatabase.py

```
#LogOnDataBase.py
#program to set up a Logon database
#Rosa 14/07/2024
import sqlite3 #use existing library#

#declare the database filename - replace later
global DBFile #create a variable - for development
DBFile = "MazeGameDatabase.db"

def Main(dbfile):
    #procedure to manage the Logon database
    CreateTable(dbfile)
    return #end procedure

****Tests****

## #add some test data
## aDML = '''INSERT INTO Avatar(AvatarFilename) VALUES("%s")''' % ("Avatar01.png")
## AddData(dbfile, aDML)

## #add some test data
## aDML = '''INSERT INTO Avatar(AvatarFilename) VALUES("%s")''' % ("Avatar02.png")
## AddData(dbfile, aDML)
##

## #add some test data
## aDML = '''INSERT INTO Avatar(AvatarFilename) VALUES("%s")''' % ("Avatar03.png")
## AddData(dbfile, aDML)
##

## #add some test data
## aDML = '''INSERT INTO Avatar(AvatarFilename) VALUES("%s")''' % ("Avatar04.png")
## AddData(dbfile, aDML)
##

## aSQL = "select * from Avatar"
## print(RunQuery(dbfile, aSQL))
##

## #add some test data
## aDML = '''INSERT INTO User(Username, Type, AvatarID)
## VALUES("%s", "%s", %i)''' % ("A_Test", "A", 3)
## AddData(dbfile, aDML)
##

## aSQL = "select * from User" #test print
## print(RunQuery(dbfile, aSQL))
## aSQL = "select * from Avatar"
## print(RunQuery(dbfile, aSQL))
## aSQL = "select * from Maze"
## print(RunQuery(dbfile, aSQL))
## aSQL = "select * from ClueSet"
## print(RunQuery(dbfile, aSQL))
## aSQL = "select * from User_Maze"
## print(RunQuery(dbfile, aSQL))
##

## #add some cluesets
## aDML = '''INSERT INTO ClueSet(ClueFilename) Values("WW1.txt")'''
```

```

## AddData(DBFile, aDML)
##
## aDML = '''INSERT INTO ClueSet(ClueFilename) Values("Animals.txt")'''
## AddData(DBFile, aDML)
##
## aDML = '''INSERT INTO ClueSet(ClueFilename) Values("Cyber.txt")'''
## AddData(DBFile, aDML)
##
## aSQL = "select * from ClueSet"
## print(RunQuery(dbfile, aSQL))
##
##
## aDML = '''INSERT INTO Maze(MazeFilename, Difficulty, MaxTime, ClueID, NickName)
VALUES("Maze1.txt", 1, "00:10:00", 3 , "Cyber Attack")'''
## AddData(DBFile, aDML)
##
## aSQL = "select * from Maze"
## print(RunQuery(dbfile, aSQL))
##
## aDML = '''INSERT INTO Maze(MazeFilename, Difficulty, MaxTime, ClueID, NickName)
VALUES("Maze2.txt", 3, "00:05:00", 2 , "Animal Farm")'''
## AddData(DBFile, aDML)
##
## aSQL = "select * from Maze"
## print(RunQuery(dbfile, aSQL))
##
## aDML = '''INSERT INTO Maze(MazeFilename, Difficulty, MaxTime, ClueID, NickName)
VALUES("Maze3.txt", 2, "00:80:00", 1, "Fun Fire Bombs History")'''
## AddData(DBFile, aDML)
##
## aSQL = "select * from User"
## print(RunQuery(dbfile, aSQL))
##
# #adding game results

#Use Case 1
#User 2"Cicero" plays "Maze1.txt" and completes it in 00:09:27, with a score of 28.

#NumEasy integer,      1- 2
#NumMedium integer,     3- 2
#NumHard integer,       5- 4

## aDML = '''INSERT INTO User_Maze(Score, TimeTaken, NumEasy, NumMedium, NumHard,
UserID, MazeID)
##          VALUES(24, "00:09:27", 4, 0, 4, 2, 4)'''
## AddData(DBFile, aDML)
## aSQL = "select * from User_Maze"
## print(RunQuery(dbfile, aSQL))

# #Use case 2:
# # "RedLamp67" plays "Maze3.txt" and completes it in 01:22:22, with a score of 75.

# aDML = '''INSERT INTO User_Maze(Score, TimeTaken, UserID, MazeID) VALUES(75,
"01:22:22", 2, 3)'''
# AddData(DBFile, aDML)
# aSQL = "select * from User_Maze"

```

```

# print(RunQuery(dbfile, aSQL))

# # Use case 3:
# # "RedLamp67" plays "Maze3.txt" and completes it in 00:16:23, with a score of 34

# aDML = '''INSERT INTO User_Maze(Score, TimeTaken, UserID, MazeID) VALUES(34,
"00:16:23", 2, 3)'''
# AddData(DBFile, aDML)
# aSQL = "select * from User_Maze"
# print(RunQuery(dbfile, aSQL))

# # Use case 4:
# # "GiraffeLeaf" plays "Maze3.txt" and completes it in 00:14:15, with a score of 57

# aDML = '''INSERT INTO User_Maze(Score, TimeTaken, UserID, MazeID) VALUES(57,
"00:14:15", 1, 3)'''
# AddData(DBFile, aDML)
# aSQL = "select * from User_Maze"
# print(RunQuery(dbfile, aSQL))

# # Use case 5:
# # "GiraffeLeaf" plays "Maze3.txt" and completes it in 00:10:48, with a score of 21

# aDML = '''INSERT INTO User_Maze(Score, TimeTaken, UserID, MazeID) VALUES(21,
"00:10:48", 1, 3)'''
# AddData(DBFile, aDML)
# aSQL = "select * from User_Maze"
# print(RunQuery(dbfile, aSQL))

# # Use case 6:
# # "GiraffeLeaf" plays "Maze3.txt" and completes it in 00:01:19, with a score of 10

# aDML = '''INSERT INTO User_Maze(Score, TimeTaken, UserID, MazeID) VALUES(10,
"00:01:19", 1, 3)'''
# AddData(DBFile, aDML)
# aSQL = "select * from User_Maze"
# # print(RunQuery(dbfile, aSQL))

# #now query multiple tables
# aQry = '''SELECT User.UserID
# FROM User
# WHERE User.UserName = "GiraffeLeaf"'''

# WantedID = RunQuery(DBFile, aQry)
# #WantedID = WantedID[0][0]

# aQry = '''SELECT User_Maze.UserID, User_Maze.MazeID, User_Maze.Score,
User_Maze.TimeTaken
# FROM User_Maze
# WHERE User_Maze.UserID = %i''' %WantedID[0][0]

# #print(RunQuery(DBFile,aQry))

#####

```

```

# aQry = '''SELECT User_Maze.UserID, User_Maze.MazeID, User_Maze.Score,
User_Maze.TimeTaken
# FROM User, User_Maze
# WHERE User.UserName = "%s"''' % "GiraffeLeaf"

# #aQry = "SELECT * FROM User_Maze"
# #print(RunQuery(DBFile,aQry))

# #find who played a particular maze "Maze3.txt"
# aQry = '''SELECT Maze.MazeID
# FROM Maze
# WHERE Maze.MazeFilename = "Maze3.txt"'''

# WantedID = RunQuery(dbfile, aQry) #[(ID, )]

# aQry = '''SELECT User_Maze.UserID, User_Maze.Score, User_Maze.TimeTaken
# FROM User_Maze
# WHERE User_Maze.MazeID = %i''' %WantedID[0][0]

# print('\n Records for "Maze3.txt":')
# GameRecords = (RunQuery(dbfile, aQry))
# print(GameRecords)

# print("\n players for those games:")

# for player in GameRecords:      #[UserID, Score, TimeTaken]
#     WantedID = player[0]      #get ID from tuple
#     #query the user table for the username
#     aQry = '''SELECT UserName
#     FROM User
#     WHERE UserID = %s''' %WantedID      #[(name, )]

#     print(RunQuery(dbfile, aQry))

## #For development, associate Cyber.txt with Maze1.txt
## aDML = '''UPDATE Maze
## SET ClueID = 3
## WHERE MazeFilename = "%s"''' % "Maze1.txt"
##
## AddData(dbfile, aDML)
##
## aQry = '''SELECT ClueID
## FROM Maze
## WHERE Maze.MazeFilename = "%s"''' % "Maze1.txt"
## AnsTable = (RunQuery(dbfile, aQry))
## aQry = '''SELECT ClueFilename FROM ClueSet WHERE ClueID = "%i"''' %AnsTable[0][0]
## print(RunQuery(dbfile, aQry))

def ConnectToDatabase(dbfile):
    #function to connect to database
    #creates it if it doesn't exist
    #returns the connection object
    #conn = a connection object
    #curs = a cursor object
    conn = sqlite3.connect(dbfile)
    curs = conn.cursor()

```

```

#return a list = one thing
return [conn, curs]

def CreateTable(dbfile):
    #Procedure to add tables to database if they are not there already

    #DDL = Data Definition Language
    #DDL script (string) to define the tables

    #Avatar Table - 2 fields
    Avatar = '''CREATE TABLE IF NOT EXISTS Avatar(
AvatarID integer primary key autoincrement,
AvatarFilename text NOT NULL
)'''

    #User Table - 4 fields
    User = '''CREATE TABLE IF NOT EXISTS User(
UserID integer primary key autoincrement,
UserName text NOT NULL,
Type text NOT NULL,
AvatarID integer NOT NULL,
FOREIGN KEY (AvatarID) REFERENCES Avatar(AvatarID)
)'''

    #ClueSet Table - 2 fields
    ClueSet = '''CREATE TABLE IF NOT EXISTS ClueSet(
ClueID integer primary key autoincrement,
ClueFilename text NOT NULL
)'''

    #Maze Table - 6 fields
    #the data type of time is time
    Maze = '''CREATE TABLE IF NOT EXISTS Maze(
MazeID integer primary key autoincrement,
MazeFilename text NOT NULL,
Difficulty integer NOT NULL,
MaxTime time NOT NULL,
ClueID integer NOT NULL,
NickName text NOT NULL,
FOREIGN KEY (ClueID) REFERENCES ClueSet(ClueID)
)'''

    #link table:
    #User_Maze Table - 10 fields
    User_Maze = '''CREATE TABLE IF NOT EXISTS User_Maze(
UMID integer primary key autoincrement,
Score integer,
TimeTaken time,
NumEasy integer,
NumMedium integer,
NumHard integer,
Accuracy text NOT NULL,
UserID integer NOT NULL,
MazeID integer NOT NULL,
COMP boolean Default 0,
FOREIGN KEY (UserID) REFERENCES User(UserID),
FOREIGN KEY (MazeID) REFERENCES Maze(MazeID)
)'''

```

```

)'''

#open the database and put the table in it
conn, curs = ConnectToDatabase(dbfile)

curs.execute(Avatar) #add tables to database - for all tables
curs.execute(User)
curs.execute(ClueSet)
curs.execute(Maze)
curs.execute(User_Maze)

conn.commit() #update the database with the new table
conn.close() #close the database

#print()
return #end procedure

def AddData(dbfile, aScript):
    #procedure to create DML aScript to add record to a table

    #print(aScript) #for testing/debugging

    #open the database
    conn, curs = ConnectToDatabase(dbfile)

    curs.execute(aScript) #add record
    conn.commit() #update the database

    conn.close() #close the database
    return #end procedure

def RunQuery(dbfile, aQry):
    #Function to execute aQry script on the database and return the Answer table

    conn, curs = ConnectToDatabase(dbfile) #call function
    curs.execute(aQry) #execute the query
    AnsTable= curs.fetchall() #fetch the results

    conn.close() #close the database
    return AnsTable #return the results

##MAIN##
Main(DBFile)

```

## PlayGame01.py

```
#PlayGame01.py
#program to play a mazegame using pygame
#Rosa 28/09/2024

#import modules to use in this file
import random      #used for random generation
import pygame      #used for writing highly portable video games
import pygame_widgets      #used for buttons, text entry etc.. in pygame
from pygame_widgets.textbox import TextBox      #specifically the text box part of
the module above
import time        #used to manipulate time and record it.
import tkinter.messagebox as mbox      #for pop up messages
import textwrap    #used to wrap text so each line is a certain number of
characters long

# Import pygame.locals for easier access to key coordinates
from pygame.locals import (
    K_UP,
    K_DOWN,
    K_LEFT,
    K_RIGHT,
    KEYDOWN,
    QUIT,
)

#####SUBPROGRAMS HERE #####

def PlayGame(aMazeRecord, aClueFile, aUserRecord, HeroAvatarFile):

    #manage time
    total_time = aMazeRecord[3]      #find the max time available
    ##      print(total_time)      #test print

    #calculate time in seconds
    hours,mins,secs = total_time.split(":")
    hrsecs = 60*60*int(hours)
    minsecs = 60*int(mins)
    secs = int(secs)
    global MaxTime
    MaxTime = hrsecs + minsecs + secs      #total game time in seconds
    ##      print(MaxTime)      #test print

    #split out info passed as parameters
    aMazeFile = aMazeRecord[1]
    #set globals
    global Username, Nickname, AvatarFile
    Username = aUserRecord[1]
    Nickname = aMazeRecord[5]
    AvatarFile = HeroAvatarFile

    Initialise(aMazeFile, aClueFile, total_time)      # set up globals and draw start
    maze
    # Function that is called to set up and manage a given maze game
    # with a given Clue set and Hero avatar
```

```

# Returns the game stats at the end

#declare global variables
global posnHero, posnSpy, posnClue, posnEmpty    # can have their values changed
global Lives, Score, TimeTaken, TimePassed      #withing this sub-program

PrevPosn = [""]*len(posnSpy)    #list to hold previous direction for each
spy(spy movement)

#set different enemy speed of different levels
if aMazeRecord[2] == 1:    #easy
    diffcount = 45
elif aMazeRecord[2] == 2:    #medium
    diffcount = 40
else:    #hard
    diffcount = 35

#set defaults
GameOver = False
paused = False
Comp = False
#make sure menu is skipped over unless called
Menu = False    #default
framecount = 0    #frame counter for spies

currenty, currentx = posnHero[0]    #get the tuple positions set out of the
list(just in case game ends without button click)

# Game timer setup
global start_time, nowtime, pause_start
start_time = time.time()    # Record the start time now
nowtime = start_time    #used to update time elapsed
pause_start = 0    # Time when the pause started

while not GameOver:    #Game over while (always run until game over)
    if not paused:    #run unless answering question or on Menu
        if posnClue == []:    #check for clue squares
            #no clue squares left
            GameOver = True    #exit while

        #Look at every event in the queue
        for event in pygame.event.get():

            # Did the user hit a key?
            if event.type == KEYDOWN:

                currenty, currentx = posnHero[0]    #get current position X and Y
                currentposn = (currenty, currentx)    #create coordinate tuple
                # Was it an arrow key to move the Hero?
                if event.key == K_UP:

##                                print("UP") #testing
                                posnEmpty.append(currentposn)    #add old position to the
empty position list.

                                yHero, xHero = posnHero[0]    # saved position as (row, col)

```

```

##                                print("\tBEFORE:\trow: %i\tcol: %i" %(yHero, xHero))
#test print

                                yHero = yHero - 1    # move up

                                posnHero[0] = (yHero, xHero)    # update position

elif event.key == K_DOWN:

##                                print("DOWN") #testing

                                posnEmpty.append(currentposn)    #add old position to the
empty position list.
                                yHero, xHero = posnHero[0]    # saved position as (row, col)
##                                print("\tBEFORE:\trow: %i\tcol: %i" %(yHero, xHero))
#test print

                                yHero = yHero + 1    # move down
                                posnHero[0] = (yHero, xHero)    # update position

elif event.key == K_RIGHT:

##                                print("RIGHT")    #test print
                                posnEmpty.append(currentposn)    #add old position to the
empty position list.
                                yHero, xHero = posnHero[0]    # saved position as (row, col)
##                                print("\tBEFORE:\trow: %i\tcol: %i" %(yHero, xHero))
#test print

                                xHero = xHero + 1    # move right
                                posnHero[0] = (yHero, xHero)    # update position

elif event.key == K_LEFT:

##                                print("LEFT")    #test print

                                posnEmpty.append(currentposn)    #add old position to the
empty position list.
                                yHero, xHero = posnHero[0]    # saved position as (row, col)
##                                print("\tBEFORE:\trow: %i\tcol: %i" %(yHero, xHero))
#test print

                                xHero = xHero - 1    # move left
                                posnHero[0] = (yHero, xHero)    # update position
else:    #any other key press
    Menu = True    #jump to Menu
    yHero, xHero = posnHero[0]    #save current X + Y in posnHero
    paused = True    #get out of the not paused if

    NewPosn = (yHero, xHero)    #tuple of updated hero position

    #what is conciquence of Hero move?
    if NewPosn in posnWall:    #can't be in wall
##                                print("Hero hit wall")    #test print

                                #Hit a wall move not allowed
                                posnHero[0] = (currenty, currentx)    #no change - revert

```

```

yHero, xHero = posnHero[0]      #reset variables
posnEmpty.remove(currentposn)    #remove update.
##                               print(posnEmpty)    #test print

elif NewPosn in posnClue:        #User will need to solve Clue
##                               print("Hero hit a Clue")    #test print
                                #Hit a clue
                                #manage pausing the game selecting a clue and marking
response
                                #remove clue whatever happens

                                posnHero[0] = NewPosn        #update new Hero position
                                #do not add to posnEmpty
                                posnClue.remove(NewPosn)      #remove Clue
                                ##                               print(posnEmpty)    #test print

                                #game over if no more clues
                                if not posnClue:
                                    paused = True            #escape not paused loop
                                    Comp = True               #game is completed
                                    paused = True            #pause to answer question

elif NewPosn in posnSpy:         #Lives will need to be edited
##                               print("Hero hit a Spy")    #test print
                                #hit a spy
                                Lives = Lives - 1
                                if Lives == 0:               #the hero is dead
                                    GameOver = True          #game is not completed
                                else:
                                    posnEmpty.remove(currentposn)    #can't teleport to
same position
                                    #use random module to choose random position
to new cell
                                    posnHero[0] = random.choice(posnEmpty)    #teleports Hero
to posnEmpty
                                    posnEmpty.append(currentposn)    #re-add old position

                                    #remove the new position of the hero after
teleportation from posnEmpty
                                    yHero, xHero = posnHero[0]
                                    NewPosn = (yHero, xHero)
                                    posnEmpty.remove(NewPosn)    #remove newPosn from
posnEmpty
                                    #print(posnEmpty)

                                elif NewPosn in posnEmpty:    #default(most occasions)
                                    #to remove old posns and keep posn Empty updated
                                    posnEmpty.remove(NewPosn)
                                    ##                               print(posnEmpty)    #test that the value has been
removed from the array.

                                #display new position of hero
##                               print("\tAFTER:\trow: %i\tcol: %i\n" %(yHero, xHero))
                                #test print

                                # Did the user click the window close button? If so, stop the loop.

```

```

elif event.type == QUIT:

##          print("QUIT")      #test print

        GameOver = True      # quit loop
        # close the window!
        pygame.quit()

        #spy movements - outside of the event if
        #set the target
        length = len(posnSpy)
        length2 = round(length/2)      #append original assignment to update when
changed
        for count in range(0,length):
            if count < (length2):      #half the spies have the hero target
                if (randspyPosn[count] != posnHero[0])and (NewPosn not in
posnWall):      #check if the Heroposn is valid
                    #change the spy position target to equal the hero position
for half/more values
                    randspyPosn[count] = posnHero[0] #only reset when necessary
                    print(randspyPosn[count])      #test print
                    print(posnHero[0])      #test print
                else:
                    #if in a wall or hit hero
                    randspyPosn[count] = (currenty, currentx)
                    print(randspyPosn[count])
                    print(posnHero[0])
                else:      #other half have random empty target
                    if randspyPosn[count] == posnSpy[count]:      #only reset once
they are equal
                        #random position
                        randspyPosn[count] =(random.choice(posnEmpty))      #random
from posnEmpty
                        #print(randspyPosn)

                    # move spies
                    #frame counter
                    framecount = framecount+1
                    #print(framecount)      #test print
                    if framecount == diffcount:      #spy movement will change dependant
on difficulty

                        #Directions for randomised
##          print(len(posnSpy))      #test print
                        for spy in range(0 , len(posnSpy)):
                            #Calculate to get from point A->B
                            StartY, StartX = posnSpy[spy]      #position of spy
                            #print(posnSpy[spy])      #test print
                            EndY, EndX = randspyPosn[spy]      #target position

                            #compare to end position
                            if (StartY < EndY):      #comparison which if true the target is
below
                                below = True
                                above = False
                                #print("below")      #test print

```

```

else:
    above = True
    below = False
    #print("above")      #test print

right
if (StartX < EndX):      #comparison which if true the target is

    right = True
    left = False
    #print("right")      #test print
else:
    left = True
    right = False
    #print("left")       #test print

#find possible paths - remove non paths
directions = ["up", "down", "right", "left"]
nwdirections = ["up", "down", "right", "left"] #ignore previous

#remove wall values
if (StartY+1, StartX) in posnWall:      #using coordinates
    directions.remove("down")
    nwdirections.remove("down")
if (StartY - 1, StartX) in posnWall:
    directions.remove("up")
    nwdirections.remove("up")
if (StartY, StartX+1) in posnWall:
    directions.remove("right")
    nwdirections.remove("right")
if (StartY, StartX-1) in posnWall:
    directions.remove("left")
    nwdirections.remove("left")

#remove directions opposite to previous directions
if PrevPosn != []:      #there won't be a previous directions

at the start.
not allowed
    #stops spies from spasming as they try to go backwards when

    if (PrevPosn[spy] == ("down")) and ("up" in directions):
        directions.remove("up")      #remove if available

    if (PrevPosn[spy] == ("up")) and ("down" in directions):
        directions.remove("down")

    if (PrevPosn[spy] == ("right")) and ("left" in directions):
        directions.remove("left")

    if (PrevPosn[spy] == ("left")) and ("right" in directions):
        directions.remove("right")

#movement
#move in the direction if the target direction is available
if above and ("up" in directions):
    direction = "up"
elif below and ("down" in directions):
    direction = "down"
elif right and ("right" in directions):

```

```

        direction = "right"
    elif left and ( "left" in directions):
        direction = "left"
    elif len(directions) == 0:          #direction is empty - dead end
        direction = random.choice(nwdirections)
    else:      #ideal direction not available
        direction = random.choice(directions)

    #to re-shuffle the spies
    #print(spy+1 < len(posnSpy))

    #to sort clumping
    if spy + 1 < len(posnSpy):
        Y, X = posnSpy[spy]
        Y1, X1 = posnSpy[spy + 1]

        # Avoid proximity clumping
        if abs(X1 - X) <= 2 and abs(Y1 - Y) <= 2:      #abs to
change the value to positive

        direction = random.choice(nwdirections)      #random
available direction - may still clup but will eventually split

    PrevPosn[spy] = direction      #set previous direction of a spy
    #print(directions, spy)      #test print

    #determin new position
    if direction == "up":          #coordinates for up
        newY = StartY - 1
        newX = StartX

    elif direction == "down":      #coordinates for down
        newY = StartY + 1
        newX = StartX

    elif direction == "left":      #coordinates for left
        newX = StartX - 1
        newY = StartY

    elif direction == "right":     #coordinates for right
        newX = StartX + 1
        newY = StartY

    #is new position in a wall
    Posn = (newY, newX)      #set coordinates

    if Posn not in posnWall:      #triple check the possibility for a
wall coordinate

        posnSpy[spy] = Posn      #update spy position
        if Posn in posnEmpty:     #sort posnEmpty list (update)
            posnEmpty.remove(Posn) #no longer empty
            posnEmpty.append((StartY, StartX))      #now empty

```

```

        if Posn in posnHero:
##            print("Spy hit a Hero")          #test print
##            print(posnHero)                  #test print
##            print()                          #test print
##            print(posnEmpty)                 #test print
            #hit a spy
            currentposn = posnHero[0]          #set current position(may
be start and not have been set)
            Lives = Lives - 1                  #increment Lives

            posnHero[0] = random.choice(posnEmpty) #teleports Hero
to New cell
            posnEmpty.append(currentposn)      #change empty

            #to remove the new position of the hero after
teleportation from posnEmpty
            yHero, xHero = posnHero[0]
            NewPosn = (yHero, xHero)
            posnEmpty.remove(NewPosn)

##            print(posnEmpty)                  #test print
##            print(posnHero)                  #test print

            if Lives == 0:                      #check Lives
                GameOver = True                #exit loop
            framecount = 0                      #reset framecount

            #update time before drawing maze
            CurrentTime = time.time()           #right now
            if int(CurrentTime-nowtime) >= 1:

                ElapsedTime = int(CurrentTime - start_time) # Elapsed time in
seconds
                nowtime = CurrentTime
                TimePassed = FormatTime(ElapsedTime)        #to sort time formatting
                #print(TimePassed)

                #is it game over?
                if ElapsedTime >= MaxTime:              #check for max time
                    GameOver = True                    #exit loop and end game
                    #game time has run out so quit loop

                DrawMaze()                             # update view

            while paused:
                #paused - questions or Menu
                framecount = -20                      #create a pause for spies before the game restarts
                global nowP                            #paused time
                pause_start = time.time()              # Note when the pause started - take of total
                nowP = pause_start

                if Menu:                                #Menu - any key press not associated with an event
                    Choice = MenuG()                  #menu gui - return button click option
                    if Choice == "RESUME":
                        Menu = False                   #continue game

```

```

        PausedTime()      #reset time
        paused = False
        break      #return to if not paused

    elif Choice == "RESTART":      #recall Play Game function
        PlayGame(aMazeRecord, aClueFile, aUserRecord, HeroAvatarFile)
    else:      #quit (exit game)
        GameOver = True
        paused = False
        break      #return to game over

if Lives == 0:      #check Lives
    GameOver = True
if not GameOver:      #answer questions if not - double check

##          print(EasyQs)      #test print
##          print(MedQs)      #test print
####          print(HardQs)      #test print

    QuestionFull = Qmanage()      #get a question
    # Display the question timer
    #declare global the question timer
    global question_time_limit
    question_time_limit = str(QuestionFull[3]) #find the question timer
##          print(question_time_limit)      #test print
string      hours,mins,secs = question_time_limit.split(":")      #split out this

    #calculate the seconds
    hrsecs = 60*60*int(hours)
    minsecs = 60*int(mins)
    secs = int(secs)
    question_time_limit = hrsecs + minsecs + secs      #total question
time in seconds
##          print(question_time_limit)      #test print

##          print(QuestionFull)      #test print
##          print(QuestionFull[0])      #test print
question      Answer = Questionpopup(QuestionFull)      #Answer input for the

    AnsCheck = ClueAns(QuestionFull, Answer)      #check the Answer

    if AnsCheck:      #correct answer
##          print(Score)      #test print
##          print(posnClue)      #test print
        if not posnClue:      # If posnClue is empty
##          print("All clues solved! Ending the game.")      #test
print

        DrawMaze()      #make the game look nice
        Comp = True      #game is completed
        GameOver = True
        paused = False
        break      #return to game over
    else:
##          print>LastQs)      #test print

```

```

        PausedTime()    #set the timer to what it was before pause
        paused = False  # Resume the game

    elif ((not AnsCheck) and (Score >= 1)):    #incorrect answer
        Score = Score - 1    #increment score
    ##        print(Score)    #test print
    ##        print(posnClue)    #test print
    #game over if no more clues
    if posnClue == []:    #check if clue squares are finished
    ##        print(posnClue)    #test print
        Comp = True
        DrawMaze()    #reset view
        GameOver = True
        paused = False
        break    #return to game over
    else:
    ##        print>LastQs)    #test print
        PausedTime()    #set game timer to pre-paused value
        paused = False    #resume game

    else:    #score = 0
        GameOver = True
        paused = False
        break    #return to game over

#Get here for game over - game Loop has quit
#display results on game for user to view
#time calculation
##    elapsed_time = time.time() - start_time
##    TimeTaken = time.strftime('%H:%M:%S', time.gmtime(elapsed_time))

##GameOver##

#stats
A = Accuracy()    #calculate accuracy
#print(A)    #test print
NumEasy = SolEasy
NumMed = SolMed
NumHard = SolHard

#stop timer
TimeTaken = TimePassed

#game over message
msg = "Your Stats Are:\n\nScore: %i\nLives: %i\nTimeTaken: %s"%(Score, Lives,
TimeTaken)
mbox.showinfo("GAMEOVER", msg)

pygame.quit()    #close game

#print([Score, Lives, TimeTaken, NumEasy, NumMed, NumHard, A, Comp])    #correct
values at this point
return[Score, Lives, TimeTaken, NumEasy, NumMed, NumHard, A, Comp]    #
endfunction - return stats for saving

```

```

def Initialise(aMazeFile, aClueFile, total_time):
    #procedure to identify global variables and set their start values
    #to draw the start maze

    #define global colours
    global colEmpty, colBG#, colHero, colSpy, colClue, colWall,(no longer needed)

    #define colours for different elements - note: replace with Avatars
    colEmpty = (255, 255, 255)          #rgb colours instead of hex tuples
    #colHero = (255, 0, 0)
    #colSpy = (68, 114, 196)
    #colClue =(237, 125, 49)          not needed anymore due to images
    #colWall =(130, 24, 144)
    colBG = "darkslategrey"          #Background colour of maze

    #declare global image variables
    global Wall, Avatar, Clue, Spy

    Wall = pygame.image.load("Wall.png")          #Load in the wall file
    Avatar = pygame.image.load(AvatarFile)          #Load in the avatar file
    Clue = pygame.image.load("Clue.png")          #Load in the Clue file
    Spy = pygame.image.load("Spy.png")          #Load in the Spy file

    #the references below are for the image icons within the game
    #ref26: https://www.deviantart.com/keatonmsk/art/Enemy-Pixel-Art-974179399
    #ref27: https://www.freepik.com/premium-vector/pixel-art-illustration-artwork-design-character-bit-icon-symbol-insect-alien-monster-video-game\_49213851.htm
    #ref28: https://www.istockphoto.com/vector/space-game-pixel-art-aliens-and-spaceship-icons-gm1263111190-369681982
    #ref29: https://forums.tigsource.com/index.php?topic=46459.0

    #declare
    global DiffQs, LastQs, EasyQs, MedQs, HardQs, Num          #to store previous
questions
    DiffQs = []
    LastQs = []
    EasyQs = []
    MedQs = []
    HardQs = []
    Num = 0

    #Accuracy Weights
    global EWeight, MWeight, HWeight
    EWeight = 0.3
    MWeight = 0.3
    HWeight = 0.4          #Hard has the largest weighing

    #global maze dimension
    global Width, Thick

    #define maze dimentions
    #for a cell
    Width = 40 #pixels (square cell)
    Thick = 2 #pixels (gap between cells)

    #create globals for the empty lists of the entity coordinates

```

```

global posnHero, posnSpy, posnClue, posnWall, posnEmpty

# create empty lists for the coordinates of the entities
posnHero = []
posnSpy = []
posnClue = []
posnWall = []
posnEmpty = []

#spy random posn (half length of spy)
global randspyPosn
randspyPosn = []

#for displaying dynamic text
global Lives, Score, TimePassed
Lives = 3
Score = 0
TimePassed = "00:00:00"

#set up the start maze configuration

global aClue
#load the Maze from the file
aMaze = LoadFile(aMazeFile)
#load the Clue from the file
aClue = LoadClueFile(aClueFile)
splitQs() #split questions by difficulty

#Populate the posn Lists and Global NumRows, NumCols
GetStartPosns(aMaze) #call procedure

#Draw the maze as a grid
#for the background
BackWidth = (NumCols * Width) + ((NumCols+1) * Thick)
BackHeight = ((NumRows+2) * Width) + ((NumRows+3) * Thick)
BackColour = pygame.Color(colBG)
#reference for the pygame colours list
#ref17: https://www.pygame.org/docs/ref/color\_list.html

#Set up a pygame window
pygame.init()
pygame.display.set_caption(Nickname)

#iterate to draw grid
global SCREEN # for game updates as progresses - global the variable SCREEN
SCREEN = pygame.display.set_mode((BackWidth, BackHeight))
SCREEN.fill(BackColour)

#Draw grid
DrawMaze() #call procedure
return #end procedure

def LoadFile(aFile):
#Function to return contents of aFile - (used for Clue and Maze file)
    try: #does try unless an error otherwise does except

        #using 'with' closes the file automatically

```

```

with open(aFile) as FileReader:

    AllLines = FileReader.readlines()    #get Lines

    #iterate to remove new Lines
    for row in range(len(AllLines)):
        AllLines[row] = AllLines[row].replace("\n", "")    #replace enters
with NULL
        #create imbedded lists for each row of the text files
    return AllLines

except:    #if an error occurs - (error message box pop up will be shown)
    mbox.showerror("File loading Failed", "Error loading file '%s'"%aFile)
##    print("ERROR with %s" %aFile)Nickname
    return ""    #NULL char

def GetStartPosns(aMaze):
    #procedure to populate the Global posn List by reading aMaze

    #changing globals can be changed
    global posnWall, posnClue, posnHero, posnSpy, posnEmpty

    global NumRows, NumCols    #will be created as global variables

    #get maze dimentions
    NumRows = len(aMaze)
    NumCols = len(aMaze[0])

    #draw the grid of cells
    for Row in range(NumRows):
        Y = (Row-1)*Thick + (Row)*Width

        #need to select cell colour for row
        aRow = aMaze[Row] #get a string

        for Col in range (NumCols):
            X = (Col-1)*Thick + (Col)*Width

            #need to select cell colour
            #upper incase the user used lower case in the file
            theChar = aRow[Col].upper()
            #selection and find coordinates
            if theChar == "W":
                posnWall.append((Row, Col))

            elif theChar == "H":
                posnHero.append((Row, Col))

            elif theChar == "S":
                posnSpy.append((Row, Col))

            elif theChar == "C":
                posnClue.append((Row, Col))

            elif theChar == "E":
                posnEmpty.append((Row, Col))

```

```

    #set original spy targets
##    print(len(posnSpy)/2) #test print
    length = len(posnSpy)
    length2 = round(length/2)          #if odd this number will be bigger than
actual half
    for count in range(0,length):      #loop as many times as the length of posnSpy
        if count < (length2):          #selection to filter out half the
results(this half is bigger on odd numbers)
            randspyPosn.append(posnHero[0])    #is initial position of hero
        else:                          #other values(other half (maybe smaller))
            randspyPosn.append(random.choice(posnEmpty))    #is any random
empty spot
##    print(randspyPosn) #test print
    return #end procedure

def DrawMaze():
    #draw the grid of cells and on screen stats
    global SCREEN #to allow for modification

    #create rectangle to overlay any temporary GUIs
    Y = ((NumRows-1)*Thick + (NumRows-3)*Width)/2    #Y coordinate of rectangle
    X = (Width + Thick)    #X coordinate of rectangle
    ExWidth = (Width*(NumCols))+(Thick*(NumCols-1))
    wid = ExWidth - (Width*2)    #width
    #print(wid) #test print
    rect = pygame.Rect(X, Y, wid, (Width*5))    #create rectangle
    CellColour = colBG    #colour the rectangle the same as the background
    pygame.draw.rect(SCREEN, CellColour, rect, 0)    #draw rectangle

    #draw Maze grid
    for Row in range(2, NumRows+2):
        Y = (Row)*Thick + (Row-1)*Width

        for Col in range (1, NumCols+1):
            X = (Col)*Thick + (Col-1)*Width

            rect = pygame.Rect(X, Y, Width, Width)

            #need to select cell colour according to precedence
            cellCoord = (Row-2, Col-1) #tuple for cell coordinate
            #selection and find coordinate

            if cellCoord in posnWall:    #add wall image to square
                #CellColour = colWall
                #pygame.draw.rect(SCREEN, CellColour, rect, 0)
                scaled_Wall = pygame.transform.scale(Wall, ((Width), (Width)))
#scale the image to the right scale
                SCREEN.blit(scaled_Wall, rect.topleft)    #Load in image

            elif cellCoord in posnSpy:    #add spy image to square with white
background
                CellColour = colEmpty    #background colour
                pygame.draw.rect(SCREEN, CellColour, rect, 0)
                scaled_Spy = pygame.transform.scale(Spy, ((Width), (Width)))
#scale the image to the right scale
                SCREEN.blit(scaled_Spy, rect.topleft)    #Load in image

```

```

        elif cellCoord in posnHero:      #add hero image to square with white
background
            CellColour = colEmpty      #background colour
            pygame.draw.rect(SCREEN, CellColour, rect, 0)
            scaled_Avatar = pygame.transform.scale(Avatar, ((Width), (Width)))
#scale the image to the right scale
            SCREEN.blit(scaled_Avatar, rect.topleft)      #Load in image

        elif cellCoord in posnClue:      #add Clue image to square
            #CellColour = colClue - no longer needed
            #pygame.draw.rect(SCREEN, CellColour, rect, 0)
            scaled_Clue = pygame.transform.scale(Clue, ((Width), (Width)))
#scale the image to the right scale
            SCREEN.blit(scaled_Clue, rect.topleft)      #Load in image

    else:      #empty squares
        CellColour = colEmpty      #pure white squares
        pygame.draw.rect(SCREEN, CellColour, rect, 0)      #Load in square

#position bar above game box - permanent on-screen variables
#variable background box creation
Y = (1)*Thick
X = (1)*Thick
ExWidth = (Width*(NumCols))+(Thick*(NumCols-1))
rect = pygame.Rect(X, Y, (ExWidth), Width)      #size of box set
CellColour = "black"
pygame.draw.rect(SCREEN, CellColour, rect, 0)      #draw rectangle on screen

#add texts fields to display data
my_ft_font = pygame.freetype.SysFont('Courier New', 30)      #style
#Title - Nickname
my_ft_font.render_to(SCREEN, (ExWidth/6, 13), (Nickname), (0, 255, 0))#position
#Username
dis = (len(Username)*19)      #right align the username - guess longest username
my_ft_font.render_to(SCREEN, (ExWidth - dis, 13), (Username), (0, 255, 0))

#add a small avatar icon
## print(AvatarFile[:6])
## print(((AvatarFile[:6]) + "0" + (AvatarFile[6:])))
A = pygame.image.load((AvatarFile[:6]) + "0" + (AvatarFile[6:]))      #Load in
the avatar file
scaled_Avatar = pygame.transform.scale(A, ((Width-10), (Width-10)))      #scale the
image to the right scale
SCREEN.blit(scaled_Avatar, (ExWidth - dis - (Width + Thick), 7))      #draw image

#Stats bar below game box - on screen variables
#variable background box creation
Y = (NumRows+2)*Thick + (NumRows+1)*Width
X = (1)*Thick
ExWidth = (Width*(NumCols))+(Thick*(NumCols-1))
rect = pygame.Rect(X, Y, (ExWidth), Width)      #size of box created
CellColour = "black"      #rectangle colour
pygame.draw.rect(SCREEN, CellColour, rect, 0)      #draw rectangle

#add texts fields to display data

```

```

my_ft_font = pygame.freetype.SysFont('Courier New', 30)    #style
my_ft_font.render_to(SCREEN, (10, Y+10), ("Score:"+str(Score)), (0, 255, 0))
my_ft_font.render_to(SCREEN, ((ExWidth/4 + 25), Y+10), ("Lives:"+str(Lives)),
(0, 255, 0))    #words and position
my_ft_font.render_to(SCREEN, ((ExWidth/4)*2 + 30, Y+10), ("Time:"+TimePassed),
(0, 255, 0))    #words and position

pygame.display.update()    #update display
return    #end procedure

def ClueAns(QuestionFull, Answer):
    #globally declare score (in order to edit)
    global Score
    QuestionA = QuestionFull[1]    #question Answer

    for event in pygame.event.get():    #stop events from piling up (should not be
necessary now with GUI screen)
        # Did the user hit a key?
        if event.type == KEYDOWN:
            #place holder to stop a build up of key presses
            Pass

    if str(QuestionA.upper()) == Answer.upper(): #answer was correct
        #print statements to test
        ##    print("True")    #test print
        ##    print(QuestionA.upper())    #test print
        ##    print(Answer.upper())    #test print
        LastQs[0] = True    #set index 0 to True if answer corect
        LastQs.append(int(QuestionFull[2]))    #append difficulty to end of array
        DiffQs.append(int(QuestionFull[2]))    #append difficulty to end of array

        #increment Score by different value depending on score
        if (int(QuestionFull[2])) == 1:    #easy
            Score = Score + 1
        elif (int(QuestionFull[2])) == 2:    #medium
            Score = Score + 3
        elif (int(QuestionFull[2])) == 3:    #hard
            Score = Score + 5
        return True    #end function return True for correct answer

    elif str(QuestionA.upper()) != Answer.upper():    #incorret answer
        #print statements to test
        ##    print(QuestionA.upper())    #test print
        ##    print(Answer.upper())    #test print
        ##    print("False")    #test print
        LastQs[0] = False    #set index 0 to false for incorrect answer
        LastQs.append(int(QuestionFull[2]))    #only append LastQs for difficulty
        return False    #end function return Fale for incorrect answer
    ##    else:
    ##        print("oh no something went wrong")    #test print
    ##        return

def LoadClueFile(Clue):
    #Function to Load and split out the parts of each question.
    #returns parts array which the imbedded question arrays.
    Clues = LoadFile(Clue)    #Load Clue file

```

```

length = len(Clues)    #get Length of Clues array
count = 0             #index for loop
Parts = []            #empty array to contain split up questions
for count in range(0, length):
    Parts.append(Clues[count].split(", "))    #split out question at , into
separate parts of the array index
    count = count+1
return Parts          #end function and return array parts

def splitQs():
    #procedure to spilt out Clueset by difficulty level
    #globally declare difficulty arrays for modification
    global EasyQs, MedQs, HardQs
    count = 0          #count index for loop

    for count in range(0, len(aClue)):
        AnsQuestion = aClue[count]    #question according to index
        #check difficulty level at index 2 and split up accordingly
        if int(AnsQuestion[2]) == 3:    #hard
            HardQs.append(AnsQuestion)
        elif int(AnsQuestion[2]) == 2:    #medium
            MedQs.append(AnsQuestion)
        elif int(AnsQuestion[2]) == 1:    #easy
            EasyQs.append(AnsQuestion)
        count = count+1    #increment count
    return    #end procedure

def pausedQmanage():
    #question selection - using precedence
    ##    print(DiffQs[-1])    #test print
    ##    print(LastQs[0])    #test print
    if not bool(LastQs[0]):    #boolean (prev answer was wrong)
        if LastQs[-1] == 3:    #prev question was hard
            #random slightly easier question (medium)
            Ans = random.choice(MedQs)
            Full = Ans
            MedQs.remove(Full)    #remove to make sure not selected again
        else:
            #random easier question (Easy)
            Ans = random.choice(EasyQs)
            Full = Ans
            EasyQs.remove(Full)    #remove to make sure not selected again

        elif ((DiffQs[-1] == 3) or (DiffQs[-1] == 2) or (EasyQs == [] and MedQs==[]) or
(MedQs == []))and HardQs:    #prev=correct (hard, medium) or (easy and medium
finished when there are Hard questions Left)
            Ans = random.choice(HardQs)    #random hard (unanswered)
            Full = Ans
            HardQs.remove(Full)    #remove to make sure not selected again
    ##        print("Hard")    #test print
    ##        print (HardQs)    #test print
        elif ((DiffQs[-1] == 1 ) or (not HardQs) or (not EasyQs)) and MedQs:
#prev=correct (easy) or (hard finished) or (easy finished) where there are Medium
questions Left.
            Ans = random.choice(MedQs)    #random medium (unanswered)
            Full = Ans
            MedQs.remove(Full)    #remove to make sure not selected again

```

```

####         print("Medium") #test print
##           print (MedQs) #test print
elif (HardQs == [] and MedQs == []) and EasyQs:      #Logic error with
returning to easy once all are solved
    Ans = random.choice(EasyQs)    #random easy (unanswered)
    Full = Ans
    EasyQs.remove(Full)    #remove to make sure not selected again
##           print("Easy") #test print
##           print (EasyQs) #test print
else:        #an error has occurred
##           print(failed) #test print
    Full = "Failed"    #this should not be possible
#global question_time_limit for modification
global question_time_limit
question_time_limit = Full[3]    #chang according to selected question
##           print(question_time_limit) #test print
return Full    #end function and return selected question

def Qmanage():
    #declared in initialise
    #global for modification
    global Num, EasyQs, MedQs, HardQs, LastQ
    #Use if for first question (allows for random selection)
    if Num == 0:
        Ans = random.choice(aClue)    #any level
        Full = Ans
    ##           print(int(Full[2])) #test print
    #remove question from related list
    if int(Full[2]) == 3:    #hard
        HardQs.remove(Full)    #remove Q
    ##           print("removed hard") #test print
    elif int(Full[2]) == 2:    #medium
        MedQs.remove(Full)
    ##           print("removed medium") #test print
    elif int(Full[2]) == 1:    #easy
        EasyQs.remove(Full)
    ##           print("removed easy") #test print
    #else:
    ##           print("Ahhhh its not working") #test print
    LastQs.append(False)    #set default first value
    Num = Num + 1    #Num no longer 0
    else:
        Full = pausedQmanage()    #evey other time besides 1st

    # the first set question is not being removed from the array.
    return Full    #end function return selected question

def Accuracy():
    #function to calculate the Accuracy at the end of the game
    #globally declare variables to save to Database at end of Game
    global SolEasy, SolMed, SolHard
    SolEasy = 0
    SolMed = 0
    SolHard = 0
    TotEasy = 0
    TotMed = 0
    TotHard = 0

```

```

if LastQs:      #at least 1 question has been answered
    LastQs.pop(0)    #to remove the status of the previous question
    #print(LastQs)      #print test for LastQs
    length = len(LastQs)    #to know how questions answered

    for questions in range(0,length):    #to find total numbers answered for
each level
        if LastQs[questions] == 1:
            TotEasy = TotEasy + 1    #to find the total number of Easy
questions asked
        elif LastQs[questions] == 2:    #medium
            TotMed = TotMed + 1

        elif LastQs[questions] == 3:    #hard
            TotHard = TotHard + 1
    #print(TotEasy)      #test Totals
    #print(TotMed)
    #print(TotHard)

    #print(DiffQs)      #Re-do for DiffQs(The correctly solved questions)
    length = len(DiffQs)

    for solved in range(0, length):    #to find numbers solved
        if DiffQs[solved] == 1:    #easy
            SolEasy = SolEasy + 1
        elif DiffQs[solved] == 2:    #medium
            SolMed = SolMed + 1
        elif DiffQs[solved] == 3:    #hard
            SolHard = SolHard + 1
    #print(SolEasy)      #test totals
    #print(SolMed)
    #print(SolHard)

    #calc Accuracy of each Lvl
    if SolEasy != 0:    #to stop errors by 0/another number
        AccEasy = SolEasy/TotEasy
        print(AccEasy)
        WAccEasy = (AccEasy * EWeight)
    else:
        WAccEasy = 0    #set if all wrong

    if SolMed != 0:    #to stop errors by 0/another number
        AccMed = SolMed/TotMed
        print(AccMed)
        WAccMed = (AccMed * MWeight)
    else:
        WAccMed = 0    #set if all wrong

    if SolHard != 0:    #to stop errors by 0/another number
        AccHard = SolHard/TotHard
        print(AccHard)
        WAccHard = (AccHard * HWeight)
    else:
        WAccHard = 0    #set if all wrong

    #calc overall accuracy
    OverallWA = WAccEasy + WAccMed + WAccHard    #final weighted accuracy

```

```

OverallWA = OverallWA * 100      #persentage
#rounding to 0dp
OverallWA = round(OverallWA)
else:
    OverallWA = 0      #if no Qs answered

OveralAcc = str(OverallWA)+ "%"      #concatinate into sting format
Acc = str(OveralAcc)      #stat to be saved
#print(Acc) #correctly printed
return Acc      #end function and return final calculated accuracy

def MenuG():
    #function to create the Menu GUI
     #(learn and experiment with widgets before question)
    #ref30 used to learn about pygame widgets
    done = False      #default
    Y = ((NumRows-1)*Thick + (NumRows-3)*Width)/2      #Y positioning of the top left
    corner of the screen
    X = (Width + Thick) #X positioning of the top left corner of the screen
    ExWidth = (Width*(NumCols))+(Thick*(NumCols-1))      #total width of the game
    wid = ExWidth - (Width*2)      #ensuring the width is the same on both sides
    # print(wid) #test print
    rect = pygame.Rect(X, Y, wid, (Width*5))      #GUI background
    CellColour = "black"      #GUI colour
    pygame.draw.rect(SCREEN, CellColour, rect, 0)      #drawn on screen (no need to
    update a it doesn't change)

    #title of mini menu gui.
    my_ft_font = pygame.freetype.SysFont('Courier New', 60)      #style (size+font)
    my_ft_font.underline = True      #underline title
    my_ft_font.render_to(SCREEN, (X+20, Y+10), ("Menu"), (0, 255, 0))      #position

    green = (0, 255, 0)      #define colour
    # Button text font - style
    button_font = pygame.freetype.SysFont('Courier New', 30)
    button_font.underline = False

    #Render text buttons square (x3)
    text1_surface, text1_rect = button_font.render(" Restart ", green)
    text2_surface, text2_rect = button_font.render(" Resume ", green)
    text3_surface, text3_rect = button_font.render(" End Game ", green)

    #Set text into rectangles (x3)
    text1_rect.topleft = (X+20, Y+90)
    text2_rect.topleft = (X+20, Y+140)
    text3_rect.topleft = (X+250, Y+140)

    #height added to buttons (x3)
    text1_rect.height += 10      # Increase height by 20 pixels
    text2_rect.height += 10
    text3_rect.height += 10

    while not done:      #GUI creation
        for event in pygame.event.get():      #check for events(key presses)
            if event.type == QUIT:      #check for quit
                done = True

    ##      print(done) #test print

```

```

# Draw buttons and rectangles
#first button
SCREEN.blit(text1_surface, text1_rect)
pygame.draw.rect(SCREEN, green, text1_rect, 2)    #draw button
#second button
SCREEN.blit(text2_surface, text2_rect)
pygame.draw.rect(SCREEN, green, text2_rect, 2)    #draw button
#third button
SCREEN.blit(text3_surface, text3_rect)
pygame.draw.rect(SCREEN, green, text3_rect, 2)    #draw button

#event handlers
if event.type == pygame.QUIT:    #top right cross
    done = True
if event.type == pygame.MOUSEBUTTONDOWN:    #mouse click
    if text1_rect.collidepoint(event.pos):
        return "RESTART"    #end function and return action wanted
    if text2_rect.collidepoint(event.pos):
        return "RESUME"    #end function and return action wanted
    if text3_rect.collidepoint(event.pos):
        return "QUIT"    #end function and return action wanted

pygame.display.update()    #update window with GUI

pygame.quit()    #Quit if done is true (quit event)

def Questionpopup(Question):
    #function to create the Question Answer GUI
    countdown = FormatQTime(question_time_limit)    #max question time

    if int(Question[2]) == 1:    #to allow the difficulty to be displayed to the
User in the title
        Difficulty = "Easy"
    elif int(Question[2]) == 2:    #medium
        Difficulty = "Medium"
    elif int(Question[2]) == 3:    #hard
        Difficulty = "Hard"

    done = False    #default
    Y = ((NumRows-1)*Thick + (NumRows-3)*Width)/2    #Y positioning of the top
left corner of the screen
    X = (Width + Thick)    #X positioning of the top left corner of the screen
    ExWidth = (Width*(NumCols))+(Thick*(NumCols-1))    #total width of the
game
    wid = ExWidth - (Width*2)    #ensuring the width is the same on both sides
    # print(wid) #test print
    rect = pygame.Rect(X, Y, wid, (Width*5))    #GUI background rectangle size
    CellColour = "black"    #GUI background colour
    pygame.draw.rect(SCREEN, CellColour, rect, 0)    #draw GUI background

    #title of mini question gui.
    my_ft_font = pygame.freetype.SysFont('Courier New', 30)    #Style (font+size)
    my_ft_font.underline = True
    #print(len>LastQs))    #test print for the question number
    my_ft_font.render_to(SCREEN, (X+20, Y+10), ("Q" + str(len>LastQs))+ " -
"+Difficulty), (0, 255, 0)) #concatinated title

```

```

my_ft_font.underline = False

#countdown timer
my_ft_font.render_to(SCREEN, ((X+wid)-175, Y+10), ("%s" %countdown), (0, 255,
0))    #on screen countdown

green = (0, 255, 0)    #colour for text
# Button text font
button_font = pygame.freetype.SysFont('Courier New', 30)    #Style (font+style)
button_font.underline = False

#ref for wrapping text after a certain number of characters
#ref31: https://stackoverflow.com/questions/49432109/how-to-wrap-text-in-pygame-using-pygame-font-font
font = pygame.font.SysFont('Courier New', 20, bold=True)    #style (font+size)
question_text = Question[0]    #text is question to be answered
txtX, txtY = X+20, Y+20    #Location of text
print(ExWidth/16)
wraplen = ExWidth/16    #number of characters before wrap
wrapper = textwrap.TextWrapper(width=wraplen)    #use imported library text
wrap
wrap_list = wrapper.wrap(text=question_text)
# Draw one line at a time further down the screen
for count in wrap_list:
    txtY = txtY + 35    #count the letters in the text
    Mtxt = font.render(f"{count}", True, green)
    SCREEN.blit(Mtxt, (txtX, txtY))    #add to screen

# Render text buttons (submit)
text1_surface, text1_rect = button_font.render(" SUBMIT ", green)

# Set text into rectangles
text1_rect.topleft = ((X+wid)- 175, Y+153)

#additional height
text1_rect.height += 10    # Increase height by 20 pixels

#create text input for Answer
base_font = pygame.font.SysFont('Courier New', 30)    #style (font+size)
user_text = ''    #current state

# create rectangle
input_rect = pygame.Rect(X+20, Y+120, 300, 36)
input_rect_color = "black"    # Input box background colour (blends in to GUI
background - not obvious)

# Restrict input Length (won't run off the page)
max_length = ExWidth/26    # Maximum characters allowed
#Guessing, estimation and testing
#print(ExWidth/63)#23 - Long
#print(ExWidth/8)#52.25 - frogs
#print(ExWidth/16)#26.125 - frogs

while not done:
    current_question_time = time.time()    #start countdown question timer
    #manage timer

```

```

#global for editing
global nowP, pause_elapse
if int(current_question_time-nowP) >= 1:    #time over
    nowP = current_question_time    #reduces number of outputs
    pause_elapse = int(current_question_time - pause_start) # Time in the
current pause
    countdown = (question_time_limit - pause_elapse)    #countdown
    countdown = FormatQTime(countdown)    #formatting
##    print(countdown)    #test print
    if question_time_limit == pause_elapse:
        return user_text    #return whatever entered at that point in time

#events in GUI
for event in pygame.event.get():    #check for events(key presses)
    if event.type == QUIT:    #check for quit
        done = True    #quit loop
        #print(done)    #test print to check quit functionality
    # Draw buttons and rectangles
    #button - submit
    SCREEN.blit(text1_surface, text1_rect)
    pygame.draw.rect(SCREEN, green, text1_rect, 2)

#event handlers
if event.type == pygame.QUIT:
    done = True
if event.type == pygame.KEYDOWN:

    # Check for backspace
    if event.key == pygame.K_BACKSPACE:    #remove previous character

        # get text input from 0 to -1 i.e. end.
        user_text = user_text[:-1]

    elif event.key == pygame.K_RETURN:    #submit text
        #print("User Input Submitted:", user_text)    #test to check
if input was correctly collected
        if user_text != "":    #stops null being accidentally
submitted
            return user_text # end function and return user input

        # Add new character if within max length
        # Unicode standard is used for string formation
        elif len(user_text) < max_length:    #max length for answers
            user_text += event.unicode

    # you can draw _input using pygame.font
    if event.type == pygame.MOUSEBUTTONDOWN:    #mouseclick (submit
        if text1_rect.collidepoint(event.pos):
            if user_text != "":    #cannot enter NULL (unless time over)
                return user_text    #end function return text

# Draw input rectangle
pygame.draw.rect(SCREEN, input_rect_color, input_rect)

# Render user text inside input rectangle
text_surface = base_font.render(user_text, True, green)
SCREEN.blit(text_surface, (input_rect.x + 5, input_rect.y + 5)) #draw text

```

```

        # Adjust input rectangle width dynamically (but keep it within bounds)
        input_rect.w = max(200, text_surface.get_width() + 10)

        pygame.draw.rect(SCREEN, green, text1_rect, 2) #redraw button (text behind
button)

        #draw a rectangle to be able to change the time
        rect = pygame.Rect((X+wid)-175, Y+10, 100, (32)) #entry rectangle size
        CellColour = "black" #to blend in with GUI background
        pygame.draw.rect(SCREEN, CellColour, rect, 0) #timer background
        #time
        my_ft_font.render_to(SCREEN, ((X+wid)-175, Y+10), ("%s" %countdown), (0,
255, 0)) #draw countdown on screen

        pygame.display.update() #update screen

    pygame.quit() #if done - quit event
    return

def FormatTime(secs):
    #function to format the seconds into hh:mm:ss for on screen display

    #hours *60*60
    hrs = secs // (60*60) #DIV (no remainder)
    secs = secs - (hrs*(60*60)) #remaining seconds
    hh = str(hrs) #string of hrs
    if hrs < 10: #add 0 to the front if less than 10
        hh = "0%s" %hh

    #mins *60
    mins = secs // 60 #DIV (no remainder)
    secs = secs - (mins*60) #remaining seconds
    mm = str(mins) #string of mins
    if mins < 10: #add 0 to the front if less than 10
        mm = "0%s" %mm

    #secs - remainder
    sec = secs #remainder = seconds
    ss = str(sec) #string of sec
    if sec < 10: #add 0 to the front if less than 10
        ss = "0%s" %ss

    Overall = "'%s:%s:%s'" %(hh,mm,ss) #concatinate for Overall
    return Overall #end function and return overall formatted time

def FormatQTime(secs):
    #function to format the seconds into mm:ss

    #mins
    mins = secs // 60 #DIV (no remainder)
    mm = str(mins) #string of mins
    if mins < 10: #add 0 to the front if less than 10
        mm = "0%s" %mm

    #secs
    sec = secs - 60*mins #remaining seconds

```

```

ss = str(sec)      #string of sec
if sec < 10:       #add 0 to the front if less than 10
    ss = "0%s" %ss

Overall = "'s:s'" %(mm,ss)    #concatinate for Overall
return Overall    #end function and return formatted overall time

def PausedTime():
    #procedure to manage paused time
    #global variable to allow for modification
    global pause_start, start_time, nowtime
    #procedure to restart the timer from pause
    pause_duration = time.time() - pause_start # How long the timer was paused
    start_time = start_time + pause_duration # Adjust the start time
    nowtime = time.time() # Reset nowtime to prevent immediate updates
    return    #end procedure

####MAIN####
##TEST INPUTS FOR DEVELOPMENT TESTS##

#set up global values and draw maze at start
#PlayGame("Maze1.txt", "Cyber.txt", "", "")

##aUserRecord = (1, 'Test', 'U', 3)
##HeroAvatarFile = "Avatar4.png"
##aMazeRecord = (1, 'Maze1.txt', 3, '00:10:00', 3, 'Cyber Attack')
##aClueFile = "Cyber.txt"
##
##PlayGame(aMazeRecord, aClueFile, aUserRecord, HeroAvatarFile)

```

43. Appendix D: Year 12 Evidence of implementing a flat file log on to a database and the subprograms to manage it. (20 Pages)

A-Level Database – Log On

## **A Level Log On Database**

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 2 of 20)**

**A-Level Database – Log On**

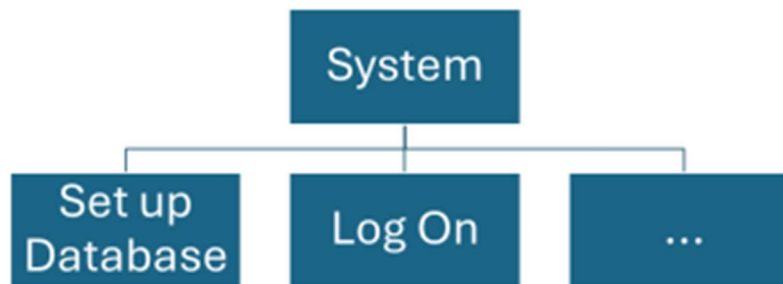
**Contents**

|                                                        |    |
|--------------------------------------------------------|----|
| Decomposition of System into a Structure Diagram:..... | 3  |
| The Database .....                                     | 3  |
| Problem .....                                          | 3  |
| Analysis.....                                          | 3  |
| Design.....                                            | 4  |
| Implementation .....                                   | 5  |
| Querying the database.....                             | 8  |
| Test .....                                             | 11 |
| Evaluation .....                                       | 14 |
| Log On.....                                            | 16 |
| Problem .....                                          | 16 |
| Analysis.....                                          | 16 |
| Design.....                                            | 17 |

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 3 of 20)**

A-Level Database – Log On

Decomposition of System into a Structure Diagram:



The database is set up at the start of the program, independently of any other module, so it can be accessed by any other module without needing to be set up each time.

### The Database

#### Problem

In Order to access the system, the user logs on with a 4 digit PIN. No greater security is needed (eg Username and Password) because there is no real personal data recorded. This could be made more secure in future if the need arises.

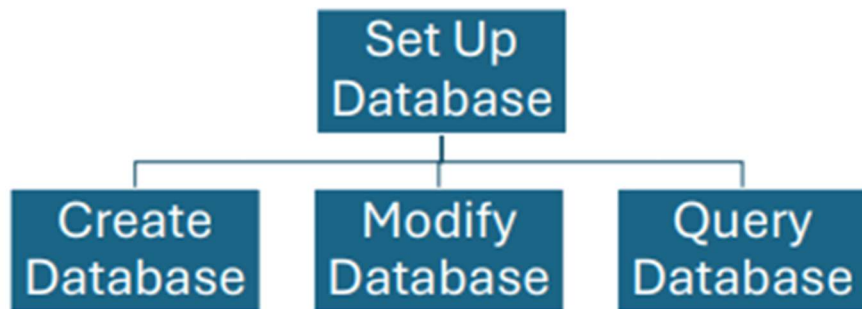
#### Analysis

1. The database, with its tables and records must be set up
2. The user table records the UserName and their 4 digit UserPIN (eg '0123' – can start with '0')
3. The database must allow:
  - a. A new user to be added – Name and PIN
  - b. The User table to be queried for a match to a given PIN, returning the Username for a match, NULL string otherwise.

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 4 of 20)**

A-Level Database – Log On

Design



By developing these as subprograms then the database can be accessed from any other module in the system.

To create the flat file (at the moment) table (ref A Level notes):

```
CREATE TABLE User
{
    UserID INTEGER NOT NULL, PRIMARY KEY, //primary key for this table
    UserName VARCHAR(10) NOT NULL,       //mandatory field up to 10 chars long
    UserPIN CHAR(4) NOT NULL              //mandatory field, fixed 4 chars long
}
```

This will create a table User with no records in it. For development and test purposes it would be useful to have some records in it.

I can specify records using Data Manipulation Language (DML) scripts:

```
INSERT INTO USER
INSERT INTO User VALUES(1, "Rosa", "1244")
INSERT INTO User VALUES(2, "Jeff", "1988")
INSERT INTO User VALUES(3, "Alice", "5555")
```

Note:UserPIN is a string because a PIN can start with 0.

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 5 of 20)**

## A-Level Database – Log On

### Implementation

```

27 def CreateTable(dbfile):
28     #Procedure to add tables to database if they are n
29
30     #DDL = Data Definition Language
31     #DDL script (string) to define the table(s)
32     User = '''CREATE TABLE IF NOT EXISTS User(
33         UserID Integer primary key autoincrement,
34         UserName text NOT NULL,
35         UserPIN Integer NOT NULL,
36     )'''
37
38     #Open the database and put the table in it
39     conn, curs = ConnectToDatabase(dbfile)
40
41     curs.execute(User) #Add table to database
42     conn.commit() #Update the database with the new ta
43     conn.close() #Close the database
44
45     print()
46     return #end procedure

```

I have written a procedure to create the database with the User table in it, if it does not exist already. This copes with running the program many times without duplicating the database.

It calls a separate ConnectToDatabase function that can be reused elsewhere.

TH procedure will be called once at the start of each program

```

16 #function to connect to database
17 def ConnectToDatabase(dbfile):
18     #creates it if it doesn't exist
19     #returns the connection object
20     #conn = a connection object
21     #curs = a cursor object
22     conn = sqlite3.connect(dbfile)
23     curs = conn.cursor()
24     #return a list = one thing
25     return [conn, curs]

```

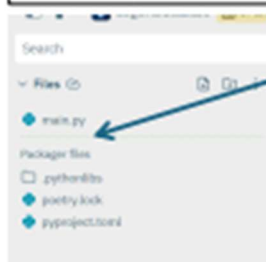
The connect to database function takes the file name as a parameter (for reuse with different filenames) and creates the database if it does not exist already.

It uses the sqlite3 database library.

Connecting to the database returns a connector object with accessor methods to manipulate the database, including creating a cursor object.

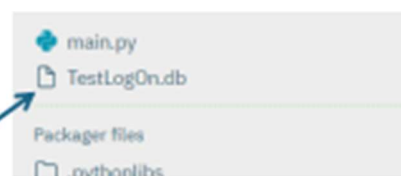
A function can only return one item, so return the conn and curs in a single line.

Ref:  
<https://www.geeksforgeeks.org/python-sqlite/> for introduction to using sqlite3, conn and curs.



Before running the code, no database.

Having run the code several times, there is still only one database



**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 6 of 20)**

### A-Level Database – Log On

```

49 def AddUser(dbfile, ID, name, pin):
50     #procedure to create DML script to add record
51     #(ID, name, pin) to User table
52
53     aDML = '''INSERT INTO User(UserID, UserName, UserPIN) VALUES(%i, "%s", %s)''' %
54     (ID, name, pin)
55
56     #open the database
57     conn, curs = ConnectToDatabase(dbfile)
58     curs.execute(aDML) #add record
59     conn.commit() #update the database
60
61     conn.close() #close the database
62     return #end procedure
63

```

I have written a new procedure AddUser, to take data values as parameters and them as a record to the database

```

def Main():
    #procedure to manage the logon database
    CreateTable(DBFile)
    AddUser(DBFile, 1, "Rosa", "2357")

```

But, when I call the procedure, I get a syntax error message from the interpreter

```

Traceback (most recent call last):
  File "/home/runner/LogOnDatabase/main.py", line 73,
in <module>
    Main(DBFile)
  File "/home/runner/LogOnDatabase/main.py", line 15,
in Main
    AddUser(DBFile, 1, "Mr Shaw", "0123")
  File "/home/runner/LogOnDatabase/main.py", line 64,
in AddUser
    curs.execute(aDML) # add the record
sqlite3.OperationalError: near "Shaw": syntax error

```

Displaying aDML, the problem is here:

```

INSERT INTO User(UserID, UserName, UserPIN) VALUES (1, Mr Shaw, 0123)
Traceback (most recent call last):

```

Modifying the aDML script, there is now no error

```

aDML = '''INSERT INTO User(UserID, UserName, UserPIN)
VALUES (%i, "%s", "%s")' % (id, name, pin)

```

```

INSERT INTO User(UserID, UserName, UserPIN) VALUES (1, "Mr Shaw", "0123")

```

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 7 of 20)**

### A-Level Database – Log On

```
INSERT INTO User(UserID, UserName, UserPIN)
VALUES (1, "Mr Shaw", "0123")
Traceback (most recent call last):
  File "/home/runner/LogOnDatabase/main.py", line 76, in <module>
    Main(DBFile)
  File "/home/runner/LogOnDatabase/main.py", line 15, in Main
    AddUser(DBFile, 1, "Mr Shaw", "0123")
  File "/home/runner/LogOnDatabase/main.py", line 67, in AddUser
    curs.execute(aDML) # add the record
sqlite3.IntegrityError: UNIQUE constraint failed: User.UserID
```

Running the code again, the aDML is correct,

But an error is reported concerning integrity and uniqueness

This is because a record with the PIN of 1 already exists.

I think the solution should be to use the autoincrement feature specified in the User table definition for the primary key:

```
# DDL script (string) to define the table(s)
User = '''CREATE TABLE IF NOT EXISTS User(
    UserID integer primary key autoincrement,
    UserName text NOT NULL,
    UserPIN text NOT NULL)'''
```

```
49 def AddUser(dbfile, name, pin):
50     #procedure to create DML script to add record
51     #(ID, name, pin) to User table
52
53     aDML = '''INSERT INTO User(UserName, UserPIN)
54     VALUES("%s", "%s")''' % (name, pin)
55     print(aDML)#print the DML statement
```

I have modified AddUser to not take an ID parameter and to not construct aDML with it

```
-- Run
INSERT INTO User(UserName, UserPIN)
VALUES ("Mr Shaw", "0123")

-- Run
INSERT INTO User(UserName, UserPIN)
VALUES ("Mr Shaw", "0123")

-- Run
INSERT INTO User(UserName, UserPIN)
VALUES ("Mr Shaw", "0123")
```

Running the code 3 times, I get no error messages so the bug I fixed and autoincrement is working.

BUT there are now multiple records in the User table with the same PIN – and all PINs must be different!

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 8 of 20)**

**A-Level Database – Log On**

**Querying the database**

In order to prevent multiple records with the same PIN, then either

- Use the PIN as the primary key – takes longer to match strings than integers,
- Query the database for a matching PIN before adding a new record.

Creating a subroutine to query the database can be reused in the actual system when adding a new user.

```
function RunQuery(dbFile, aQry)

    # Executes aQry on the database and returns the Answer Table

    conn, curs = ConnectToDatabase(dbfile) // call existing function

    curs.execute(aQry) // run the query

    AnsTable = curs.fetchall() // get the answer table

    conn.close() // close database when finished

    return AnsTable // Will be empty for no match, have at least one value
    in otherwise

endfunction
```

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 9 of 20)**

### A-Level Database – Log On

```
70 def RunQuery(dbfile, aQry):
71     #Function to execute aQry script on the
       database and return the Answer table
72
73     conn, curs = ConnectToDatabase(dbfile) #call
       function
74     curs.execute(aQry) #execute the query
75     AnsTable= curs.fetchall() #fetch the results
76
77     conn.close() #close the database
78     return AnsTable #return the results
```

I have coded the function.

```
14 #query the database to confirm the contents
15 print(RunQuery(DBFile, '''SELECT * FROM
       User'''))
16
```

I have queried the User table to show all records

Run

Ask AI

132ms on 11:19:13, 06/07 ✓

```
[(1, 'Rosa', 2357), (2, 'Rosa', 2357), (3, '
Rosa', 2357), (4, 'Rosa', 2357), (5, 'Rosa',
2357), (6, 'Rosa', 2357), (7, 'Rosa', 2357)
]
```

The database and query are working – clearly there are multiple records with the same PIN!

The approach now should be to check the database to see if the PIN is in use already for uniqueness. If it is, do not add the record. If the PIN is not in use (answer table is empty) then add the record in the User table.

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 10 of 20)**

### A-Level Database – Log On

Algorithm to add a new record to the database if the PIN is unique

**Procedure AddRecord(dbfile, name, pin)**

//procedure to add a new record if the pin is unique

Qry = "SELECT UserName FROM User WHERE UserPIN = pin" //create

script

If RunQuery(dbfile, Qry) then //record already exists

Print("PIN already in use – try again")

Else //no matching PIN

AddData(dbfile, name, pin) //call procedure

Print("New record added to database")

Endif

**Endprocedure**

```

80 def AddRecord(dbfile, name, pin):
81     #procedure to create DML script to add record
82     Qry = '''SELECT UserName FROM User WHERE UserPIN = "%s"''' % pin
83
84     if RunQuery(dbfile, Qry): #pin already in use
85         print("PIN '%s' already in use – try again" % pin)
86     else:
87
88         AddUser(dbfile, name, pin) #add the record
89         print("User '%s' added to database" % name)
90
91     return #end procedure
    
```

I have coded the procedure as per its design.

```

def Main():
    #procedure to manage the logon database
    CreateTable(DBFile)

    print("\n part 1 (record with existing pin) \n")
    #add some default records
    AddUser(DBFile, "Rosa", "2357")
    AddRecord(DBFile, "Vira", "2357")

    #query the database to confirm the contents
    print(RunQuery(DBFile, '''SELECT * FROM User'''))

    print("\n part2: (unique pin used) \n")
    AddRecord(DBFile, "Alice", "9385") #try to use a different PIN

    #query the database to confirm the contents
    print(RunQuery(DBFile, '''SELECT * FROM User'''))

    return #end of procedure
    
```

To test Procedure:

1. Try to add a record with a new name but existing PIN
2. Try to add a new record with a new name and unique PIN

```

part 1 (record with existing pin)
PIN '2357' already in use – try again
[(1, 'Rosa', 2357), (2, 'Rosa', 2357), (3, 'Rosa', 2357), (4, 'Rosa', 2357),
(5, 'Rosa', 2357), (6, 'Rosa', 2357), (7, 'Rosa', 2357), (8, 'fish', 519)]

part2: (unique pin used)
User 'Alice' added to database
[(1, 'Rosa', 2357), (2, 'Rosa', 2357), (3, 'Rosa', 2357), (4, 'Rosa', 2357),
(5, 'Rosa', 2357), (6, 'Rosa', 2357), (7, 'Rosa', 2357), (8, 'fish', 519),
(9, 'Alice', 9385)]
    
```

The record with a PIN matching one already in the database is not added.

The record with a unique PIN is added to the database, with the allocated primary key auto incremented

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 11 of 20)**

A-Level Database – Log On

**Evaluation:**

The User database code is complete as per the analysis – the database is created, the User table is added and the default records can be added. The database can be queried.

Q: Should validation subprograms now be added since the UserName and UserPIN fields are specified as NOT NULL (ie mandatory):

- UserName: not NULL string (presence check)
- UserPIN:
  - o Not NULL strings (presence check)
  - o 4 chars long (length check)
  - o Each char should be a digit (type check)

```
Function AreValidValues(name, pin)
//function to return True if name & pin are valid, false otherwise
If (name == "") or (pin == "") then
    Return False //either or both are NULL strings
Else //validate pin
    If pin.length != 4 then
        Return False //fails length check
    Else //type check
        For indx = 0 to 3 //4 digit pin expected
            If pin.Substring(indx, 1) not in digits then
                //not a digit
                Return False //quit – failed validation
            Endif
        Next indx
    Endif
    //only get here if both valid
    Return True
Endfunction
```

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 12 of 20)**

### A-Level Database – Log On

```
115 = def isValidName(name, pss):
116     """function to return true if name did not contain special characters, false otherwise
117     """
118     if name == "" or len(name) == 0:
119         return False
120     return True
121
122 = def isValidPss(pss):
123     """function to return true if pss was valid, false otherwise
124     """
125     if len(pss) != 8:
126         return False #failed length check
127     #check if pss is a string of 8 characters
128     for char in pss:
129         if char not in "0123456789":
130             return False #failed type check
131     #if we get here, the pss is valid
132     return True
```

I have coded a function as  
per the design

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 13 of 20)**

**A-Level Database – Log On**

**Test**

Test Plan for validation

Test with:

- valid(in acceptable range).
- bondary(at extremes of range, but still valid).
- invalid(out of range).
- erroneous data(wrong data type).

| <u>No.</u> | <u>Description</u>                                           | <u>Input Data</u>                    | <u>Expected Outcome</u>                  | <u>Actual outcome</u> | <u>P/F</u> |
|------------|--------------------------------------------------------------|--------------------------------------|------------------------------------------|-----------------------|------------|
| <u>1</u>   | <u>Valid Name and PIN are accepted and record is created</u> | <u>"William"</u><br><u>"4368"</u>    | <u>True – William added to database</u>  |                       |            |
| <u>2</u>   | <u>Invalid Name and PIN Are Rejected</u>                     | <u>""</u><br><u>""</u>               | <u>False – not added to database</u>     |                       |            |
| <u>3</u>   | <u>Invalid Name Valid PIN Are Rejected</u>                   | <u>""</u><br><u>"9420"</u>           | <u>False – not added to database</u>     |                       |            |
| <u>4</u>   | <u>Valid Name Invalid PIN Are Rejected</u>                   | <u>"Susan"</u><br><u>""</u>          | <u>False – not added to database</u>     |                       |            |
| <u>5</u>   | <u>Valid Name Invalid PIN (too short) Are Rejected</u>       | <u>"Riley"</u><br><u>"402"</u>       | <u>False – not added to the database</u> |                       |            |
| <u>6</u>   | <u>Valid Name Invalid PIN (too long) Are Rejected</u>        | <u>"Ellie"</u><br><u>"022502340"</u> | <u>False – not added to the database</u> |                       |            |
| <u>7</u>   | <u>Valid Name Invalid PIN (erroneous) Are Rejected</u>       | <u>"Jeff"</u><br><u>"kf24"</u>       | <u>False – not added to the database</u> |                       |            |

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 14 of 20)**

### A-Level Database – Log On

```

136 def TestValidation(dbfile, name, pin):
137     #procedure to test validation
138     #try to add valid and invalid data
139
140     if AreValidValues(name, pin):
141
142         AddRecord(dbfile, name, pin)
143     else:
144         print("ERROR. Invalid Name (%s) or Invalid pin (%s)" % (name, pin))
145
146     return #end of procedure

```

I have coded a procedure to support automatic testing of the AreValidValues function

```

25 print(RunQuery(dbfile, "SELECT * FROM User"))
26 print()
27
28 TestValidation(dbfile, "William", "4368") #valid values
29 TestValidation(dbfile, "", "") #invalid values
30 TestValidation(dbfile, "", "9420") #invalid values
31 TestValidation(dbfile, "Susan", "") #invalid values
32 TestValidation(dbfile, "Riley", "402") #invalid values
33 TestValidation(dbfile, "Elle", "022502340") #invalid values
34 TestValidation(dbfile, "Jeff", "kf24") #invalid values
35
36 #query the database to confirm the contents
37 print()
38 print(RunQuery(dbfile, "SELECT * FROM User"))
39

```

I have added calls to the procedure using the test data values.

I have queried the database before and after the test to confirm if records are added or not

```

User 'William' added to database
ERROR. Invalid Name () or Invalid pin ()
ERROR. Invalid Name () or Invalid pin (9420)
ERROR. Invalid Name (Susan) or Invalid pin ()
ERROR. Invalid Name (Riley) or Invalid pin (402)
ERROR. Invalid Name (Elle) or Invalid pin (022502340)
ERROR. Invalid Name (Jeff) or Invalid pin (kf24)

```

After the tests, William is now in the database

```

[(1, 'Rosa', 2357), (2, 'Rosa', 2357), (3, 'Rosa', 2357), (4, 'Rosa', 2357),
(5, 'Rosa', 2357), (6, 'Rosa', 2357), (7, 'Rosa', 2357), (8, 'fish', 519),
(9, 'Alice', 9395), (10, 'Alice', 9315), (11, 'William', 4368)]

```

Tests: 1 2 3 4 5 6 7

The validation works as intended and only new values are added as new values in the database.

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 15 of 20)**

## A-Level Database – Log On

### Importing the Database Code as Library

Modularising allows me to develop programs that can be imported as a library into other programs, making them less complex (only focuses on one task) and not so long (hard to debug)

```
1 #UseDatabase.py
2 #Short program to import the database as a library and access
3 #Rosa 18/06/2024
4 from LogOnDatabase import * as a library
5 from TestValidation import * as a library
6
7 #See what can be accessed
8 #Call the main procedure
9 print()
10 print(DEFFile) #See the database filename
11 print()
12
13 #Try to add a new record
14 print(TestValidation(DEFFile, "Brian", "2354")) #See the data
15 print() #Blank line
16
17
18 #See all contents of database
19 print(RunQuery(DEFFile, "SELECT * FROM User"))
20 print()
```

I have written a short program to import the database code as a library and try to:

1. See the database filename
2. Add a new record
3. See the contents of the database

```
PIN '4368' already in use - try again
ERROR. Invalid Name () or invalid pin ()
ERROR. Invalid Name () or invalid pin (9420)
ERROR. Invalid Name (Susan) or invalid pin ()
ERROR. Invalid Name (Riley) or invalid pin (402)
ERROR. Invalid Name (Ellie) or invalid pin (82250
2348)
ERROR. Invalid Name (Jeff) or invalid pin (kf24)

[(1, 'Rosa', 2357), (2, 'Rosa', 2357), (3, 'Rosa',
2357), (4, 'Rosa', 2357), (5, 'Rosa', 2357), (6,
'Rosa', 2357), (7, 'Rosa', 2357), (8, 'fish', 5
19), (9, 'Alice', 9305), (10, 'Alice', 9315), (11,
'William', 4368), (12, 'Brian', 6603), (13, 'Br
ian', 2503)]
```

The Main() procedure in LogOnDatabase is called when the library is imported, which is running previous tests – remove/comment out!

```
TestLogOn.db
User 'Brian' added to database
None

[(1, 'Rosa', 2357), (2, 'Rosa', 2357), (3, 'Rosa',
2357), (4, 'Rosa', 2357), (5, 'Rosa', 2357), (6,
'Rosa', 2357), (7, 'Rosa', 2357), (8, 'fish', 5
19), (9, 'Alice', 9305), (10, 'Alice', 9315), (11,
'William', 4368), (12, 'Brian', 6603), (13, 'Br
ian', 2503), (14, 'Brian', 2354)]
```

UseDatabase does import LogOnDatabase as a library correctly and can access its member functions

## A-Level Database – Log On

### Evaluation

### Log On

#### Problem

A user accesses a system by logging on with a 4 digit PIN.

This is compared to a PIN in their database account and the log on attempt is accepted or rejected.

Up to 3 log on attempts are allowed.

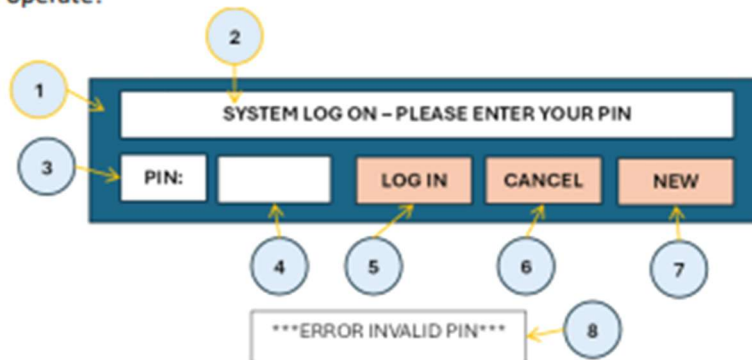
**Analysis** -> Numbered Success Criteria (what the solution must demonstrate)

1. The user logs on with a 4 digit PIN
2. The PIN must be validated (meets the rules, 'as expected')
  - a. presence check (something entered?)
  - b. length check (4 chars – eg "0123")
  - c. Type check – all digit chars
3. If the PIN fails validation then display an appropriate error message
4. After successful validation, the PIN is authenticated – a matching record is searched for in the database
5. If successful, greet the user
6. If unsuccessful, display an appropriate error message
7. Up to 3 logon attempts are allowed

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 17 of 20)**

**A-Level Database – Log On**

**Design** – based on the success criteria, what will the solution look like and operate?



| Item number | Identifier    | Description                                                                                                                  |
|-------------|---------------|------------------------------------------------------------------------------------------------------------------------------|
| 1           | frmLogOn      | <b>Frame</b> - container to hold all the necessary log on widgets<br>Moving the Frame groups the widgets so they all move to |
| 2           | lblLogOn      | <b>Label</b> - read only title to display what part of the system is using                                                   |
| 3           | lblPIN        | <b>Label</b> - read only to show what is expected to be entered into the text box                                            |
| 4           | txtPIN        | <b>Text box</b> - for the user to enter / type their PIN                                                                     |
| 5           | cmdLogOn      | <b>Command button</b> - easy for the user to click on to initiate logging on after entering a PIN                            |
| 6           | cmdLOCancel   | <b>Command button</b> - easy for the user to click on to quit the program                                                    |
| 7           | cmdNewAccount | <b>Command button</b> - easy for the user to click and go to another GUI where they can create an account                    |
| 8           | msgLogOn      | <b>Message box</b> for LogOn messages:<br>***ERROR INVALID PIN***<br>***ERROR USER NOT FOUND***<br>***Hello Rosannah***      |

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 18 of 20)**

**A-Level Database – Log On**

**Appendix A: Database source code**

```
#LogOnDataBase.py
# program to set up a logon database
#Rosa 05/06/2024
import sqlite3 #use existing library#

#declare the database filename - replace later
DBFile = "TestLogOn.db" #create a variable

def Main():
    #procedure to manage the logon database
    CreateTable(DBFile)

    #print("\n part 1 (record with existing pin) \n")
    #add some default records
    #AddUser(DBFile, "Rosa", "2357")
    #AddRecord(DBFile, "Vira", "2357")

    #query the database to confirm the contents
    #print(RunQuery(DBFile, '''SELECT * FROM User'''))

    #print("\n part2: (unique pin used) \n")
    #AddRecord(DBFile, "Alice", "9305") #try to use a different PIN

    #query the database to confirm the contents
    print(RunQuery(DBFile, '''SELECT * FROM User'''))
    print()

    TestValidation(DBFile, "William", "4368") #valid values
    TestValidation(DBFile, "", "") #invalid values
    TestValidation(DBFile, "", "9420") #invalid values
    TestValidation(DBFile, "Susan", "") #invalid values
    TestValidation(DBFile, "Riley", "402") #invalid values
    TestValidation(DBFile, "Ellie", "022502340") #invalid values
    TestValidation(DBFile, "Jeff", "kf24") #invalid values

    #query the database to confirm the contents
    print()
    print(RunQuery(DBFile, '''SELECT * FROM User'''))

    return #end of procedure

#function to connect to database
def ConnectToDatabase(dbfile):
    #creates it if it doesn't exist
    #returns the connection object
    #conn = a connection object
    #curs = a cursor object
    conn = sqlite3.connect(dbfile)
    curs = conn.cursor()
    #return a list = one thing
    return [conn, curs]
```

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 19 of 20)**

**A-Level Database – Log On**

```
def CreateTable(dbfile):
    #Procedure to add tables to database if they are not there already

    #DDL = Data Definition Language
    #DDL script (string) to define the table(s)
    User = '''CREATE TABLE IF NOT EXISTS User(
        UserID integer primary key autoincrement,
        UserName text NOT NULL,
        UserPIN integer NOT NULL
    )'''

    #open the database and put the table in it
    conn, curs = ConnectToDatabase(dbfile)

    curs.execute(User) #add table to database
    conn.commit() #update the database with the new table
    conn.close() #close the database

    print()
    return #end procedure

def AddUser(dbfile, name, pin):
    #procedure to create DML script to add record
    #(ID, name, pin) to User table

    aDML = '''INSERT INTO User(UserName, UserPIN) VALUES("%s", "%s")''' % (name,
pin)

    #print(aDML)#print the DML statement

    #open the database
    conn, curs = ConnectToDatabase(dbfile)

    curs.execute(aDML) #add record
    conn.commit() #update the database

    conn.close() #close the database
    return #end procedure

def RunQuery(dbfile, aQry):
    #Function to execute aQry script on the database and return the Answer table

    conn, curs = ConnectToDatabase(dbfile) #call function
    curs.execute(aQry) #execute the query
    AnsTable= curs.fetchall() #fetch the results

    conn.close() #close the database
    return AnsTable #return the results
```

**Appendix C: Year 12 Evidence of implementing a flatfile log on to a database and the subprograms to manage it. (Page 20 of 20)**

**A-Level Database – Log On**

```
def AddRecord(dbfile, name, pin):
    #procedure to create DML script to add record
    Qry = '''SELECT UserName FROM User WHERE UserPIN = "%s" ''' %pin

    if RunQuery(dbfile,Qry): #pin already in use
        print("PIN '%s' already in use - try again" %pin)
    else:

        AddUser(dbfile, name, pin) #add the record
        print("User '%s' added to database" %name)

    return #end procedure

def AreValidValues(name, pin):
    #function to return true if name AND pin are valid, false otherwise
    if (name == "") or (pin == ""):
        #failed presence check
        return False
    else:
        #passed presence check - name done
        #now validate pin
        if len(pin) != 4:
            return False #failed length check
        else:
            digits = "0123456789" #string of char
            #iterate char by char through pin

            for indx in range(4):
                if pin[indx] not in digits:
                    return False #failed type check

            #only get here for valid name and pin
            return True

def TestValidation(dbfile, name, pin):
    #procedure to test validation
    #try to add valid and invalid data

    if AreValidValues(name, pin):

        AddRecord(dbfile, name, pin)
    else:
        print("ERROR. Invalid Name (%s) or invalid pin (%s)" % (name, pin))

    return #end of procedure

#Main - call the subprograms in the correct order
Main()
```